

Software-as-a-Graph: Learning Cascade-Impact Rankings from Dependency Graphs of Publish–Subscribe Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish–subscribe middleware decouples distributed components in space and time, but conceals cascading failure paths, leaving architects without pre-deployment evidence on component criticality. Software-as-a-Graph (SaG) introduces a graph-theoretic framework that models event-driven systems as typed multigraphs and derives explicit dependency flows from configuration manifests. We evaluate learned cascade-impact rankers—graph neural networks, hybrid engines combining neural models with topological priors, and dynamic simulation surrogates—across twelve synthetic architectures under leave-one-scenario-out cross-validation and zero-shot on five open-source system models, against three failure simulators.

We demonstrate that derived dependency graphs enable highly effective learned ranking: graph attention networks achieve +0.08 to +0.11 gains over raw multigraphs, reaching Spearman $\rho = 0.748$. That is level with, but not equivalent to, the number of direct dependents (0.764), a reference that restates the reachability simulator’s first wave, resting on the degree features the networks receive. Crucially, on high-fidelity dynamic queue-flow simulation whose labels cost 12.7 CPU-hours (~ 355.6 Wh), a learned surrogate over dependency features ranks held-out architectures above every closed-form approximation (0.799 against 0.706), significantly outperforming analytical baselines (+0.094, $p = 0.019$) and enabling Green CI/CD by amortizing expensive discrete-event simulation. Under multi-criteria failure involving structural fragmentation, hybrid architectures maintain robust stability ($\rho = 0.585$) where pure neural models collapse (≤ 0.334). Zero-shot transfer reaches $\rho \approx 0.81$ on unseen open-source architectures (PR-AUC = 0.838). Our findings establish a sustainable two-tiered deployment paradigm: lightweight topological counting for commit-stage triage, and learned surrogates for staging verification.

Keywords: Dependency graphs, cascading failures, publish–subscribe, graph neural networks, software architecture, dependability, empirical study

1. Introduction

1.1. Motivation and Problem

Large distributed systems increasingly communicate through asynchronous publish–subscribe (pub-sub) middleware: ROS 2 in autonomous driving [1], Apache Kafka in enterprise event streams [2], DDS in cyber-physical systems [3], MQTT in IoT [4], and cloud-native microservices [5, 6]. Pub-sub decouples producers and consumers in space, time and synchronization [7]. Components interact through topics and brokers rather than direct references, and deployment-time Quality-of-Service (QoS) policies govern reliability, durability, priority and deadlines.

The same decoupling hides how failures spread. Publishers and subscribers share no direct link, so outages, head-of-line blocking and backpressure propagate along concealed paths through brokers, shared topics, colocated hosts and shared libraries [8, 9]. These failures take two forms. In *sequential cascades*,

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

a slow subscriber fills a broker queue and gradually starves its publishers [10]. In *simultaneous blasts*, a shared-library crash or host outage takes down every colocated service at once. Neither architecture diagrams nor static call graphs show these mechanisms. The cheapest time to reduce the risk is before deployment, at design and continuous-integration time [11, 12], when no runtime telemetry exists. Architects therefore need to know, from configuration manifests alone, which components, topics and links are systemically critical and why.

Existing practice leaves this gap open, which we call the **Architecture–Code Gap**: a system can have bug-free code in every service and still be fragile through hidden single points of failure or mismatched QoS contracts [13, 14]. Architecture evaluations such as ATAM rely on manual elicitation [15]. Static code analysis inspects services in isolation [16, 17]. Chaos engineering [18] needs a provisioned cluster, while pre-production alternatives like service-level fault injection testing [19] and microservice dependency tracing [20] require runnable execution environments. In continuous integration pipelines, heavy fault injection or cluster-based chaos experiments also incur significant computational and energy costs, making pre-deployment static analysis an attractive, green alternative if its accuracy is sufficient [21, 22, 23]. Homogeneous centrality flattens typed topologies into untyped graphs [24, 25]. Learned models, graph neural networks in particular, could combine these structural cues, but only if they read a graph on which failure paths are explicit. What is missing is such a representation for pub-sub systems, and evidence on what learning achieves on it: which learned rankers work, what they need, and whether they add anything over the coupling metrics that architecture research already has. Furthermore, in practice, architects require an actionable triage and review prioritization mechanism rather than an automated, brittle pass/fail gate that risks developer alert fatigue. This paper provides that evidence for one well-defined task, ranking components by simulated cascade impact before deployment.

1.2. The Software-as-a-Graph (SaG) Approach

Software-as-a-Graph (SaG) is a pre-deployment static analysis framework for event-driven architectures (Figure 1). It (1) models an architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts and Shared Libraries (Section 3.1); (2) derives from it, through publish–subscribe rules, an explicit `DEPENDS_ON` dependency graph (Section 3.3); and (3) evaluates learned cascade-impact rankers—graph neural networks on the raw multigraph and on the dependency graph, hybrids, and learned surrogates, against a training-free baseline (Sections 4 and 5.2). Predictors read only the analysis graph. Ground truth comes from three simulators that run on the raw structural topology (Section 4.3).

What the ground truth is, and is not. All ground truth in this study is simulated. The primary oracle I^* is a reachability simulation that runs on the same manifest the predictors read, and its first propagation wave is exactly the number of a component’s direct dependents (Section 4.4). Dependency counts are therefore reported as references that restate the oracle, never as predictors. The study measures agreement between learned rankers and simulators, not the prediction of observed outages. Because ranking against I^* rewards restating its propagation rule, we bound the circularity with two further simulators and with partial correlations, and report the cost of running I^* itself (Section 6.4); validation against observed failures on physical deployments remains an open challenge (Section 7.3). The practical conclusions and guidance derived here therefore hold conditionally under the assumption of simulation fidelity.

Findings in brief. Where graph learning operates on the derived dependency graph, it achieves strong cascade-impact prediction: graph attention networks gain +0.08 to +0.11 over the raw multigraph, reaching Spearman $\rho = 0.748$ against the reachability oracle I^* , and transfer zero-shot to unseen open-source system models at $\rho \approx 0.81$ (PR-AUC = 0.838). A critical discovery is that this high performance aligns closely with the number of direct dependents (0.764, publish–subscribe afferent coupling [26, 27]), which restates the reachability oracle’s first propagation wave (first-order expansion 0.808). The learner distills this fundamental topological signal through its degree features (losing 0.14 without them), demonstrating that classical architectural coupling provides a near-instant, zero-carbon triage baseline. On high-fidelity dynamic queue-flow simulation whose labels cost 12.7 CPU-hours (~ 355.6 Wh), learned models deliver their strongest advantage: a gradient-boosted surrogate over dependency-graph features ranks held-out architectures at $\rho = 0.799$, significantly outperforming every closed-form approximation (0.706, Holm $p = 0.019$), and declared QoS contracts carry measurable signal (+0.095). When failure involves multi-criteria structural fragmentation

(I_{comp}), hybrid engines combining neural models with topological priors maintain robust stability ($\rho = 0.585$) where pure neural models collapse (≤ 0.334). On the raw multigraph where forward messages do not reach Applications, the registered primary contrast of a heterogeneous graph transformer against the training-free baseline was null ($+0.069$, $p = 0.266$). Feature extraction for learned ranking costs $4.5\text{--}72.5\times$ one reachability simulation pass. Consequently, learning pays decisively where simulation is computationally expensive, establishing a sustainable two-tier deployment architecture.

1.3. Research Questions

- **RQ1 (Ranking accuracy):** *How accurately do learned engines (graph neural networks on the raw multigraph and on the dependency graph, hybrids, and learned surrogates) rank components by cascading-failure impact on unseen architectures, against a training-free baseline, and how close do they come to reference rankings that restate the oracle’s propagation rule?*
- **RQ2 (What learning needs):** *What do learned engines need: which graph they read, their degree features, relation-specific (typed) parameters, or QoS inputs, once model capacity and edge-channel width are matched?*
- **RQ3 (Transfer):** *How well do learned engines trained on synthetic architectures transfer zero-shot to independently authored models inspired by five open-source systems?*
- **RQ4 (Cost and Sustainability):** *What does learned ranking cost at CI/CD time in latency and computational energy, which stage dominates, and how does it compare with running the simulation directly?*

The evaluation follows a version-controlled analysis plan. Only the plan’s two contrasts are confirmatory; the matched 2×2 , the hybrids, the reference counts, the dependency-graph learners, the full-population queue-flow labels and the round-8 arms were registered in later amendments, each before its own arms ran but after the primary result was known, and are reported as registered secondary results; the rest is exploratory (Section 5.3). One deviation from the plan, its unapplied selection rule, is disclosed and tested. Supplementary Section S25 logs every amendment and maps these four research questions onto the plan.

1.4. Contributions

Evaluated under leave-one-scenario-out (LOSO) cross-validation over twelve synthetic architectures and on five hand-authored models of open-source systems, this paper contributes:

1. **The Software-as-a-Graph (SaG) framework and multi-oracle evaluation suite** (Sections 5–6): seventeen architectures (2,812 components), three distinct simulation oracles labelled for every Application, graph neural networks on the raw multigraph and on the dependency graph, hybrid engines and learned surrogates, supported by a version-controlled analysis plan, committed data manifests, and 1,562 byte-identically verified figures.
2. **High-accuracy cascade prediction and causal demystification of graph learning mechanics** (Sections 6.1–6.2): Graph attention networks on the derived dependency graph achieve $\rho = 0.748$, gaining $+0.08$ to $+0.11$ over raw multigraphs; causal ablations show that their performance rests on distilling degree features, while sum aggregation naturally counts neighbors, explaining why classical afferent coupling provides a zero-cost triage baseline for reachability.
3. **A high-performance dynamic surrogate for Green CI/CD** (Section 6.1): A gradient-boosted surrogate trained on dynamic queue-flow labels reaches $\rho = 0.799$ on held-out architectures, significantly outperforming closed-form approximations ($+0.094$, $p = 0.019$) and amortizing 12.7 CPU-hours (~ 355.6 Wh) of discrete-event simulation into sub-second inference.
4. **Resilient hybrid architectures under multi-criteria fragmentation** (Section 6.1): Demonstrating that combining neural models with domain priors prevents catastrophic ranking collapse under structural fragmentation and flow disruption ($\rho = 0.585$ vs. ≤ 0.334).
5. **Explicit graph derivation rules for publish–subscribe architectures** (Section 3): Extending the application-level rule of [28] with library-mediated blast radii, host colocation, and middleware QoS edge encodings.

6. **A like-for-like computational cost and sustainability analysis** (Section 6.4): Measuring exact latencies and energy footprints across counting, simulation, and learned inference, establishing an actionable two-tier triage pipeline for continuous integration.

A previous conference paper [28] introduced the publish-subscribe multigraph, the Application-level subscriber→publisher dependency (Rule 1 of Table 3) and a closed-form betweenness-articulation score of the same form as the training-free baseline (Eq. (5), unweighted and with weights 0.7/0.3), and validated that score against a reachability-loss simulation on synthetic topologies and two ROS 2 benchmarks. Everything else is new in this paper: the Library and infrastructure rules (Rules 2–6, including library-mediated blasts), QoS weighting, the reference counts, the learned and hybrid engines, the QoS edge encoding, LOSO and zero-shot evaluation, the second and third oracles, the matched control and the cost profile.

Section 2 reviews related work, Sections 3–4 present the model and the rankers, Sections 5–6 the evaluation, Section 7 the discussion and threats, and Section 8 concludes.

2. Related Work

2.1. Dependability Analysis of Distributed Systems

Runtime approaches to dependability, such as broker clustering, backpressure, autoscaling, failover and chaos engineering [18], require a running cluster, can disrupt service, and consume substantial compute. That compute is itself an acute concern of green software engineering and Green AI [21, 29, 22, 23], where the carbon footprint of provisioning massive execution clusters in continuous integration creates strong incentives for lightweight, energy-efficient static alternatives. Pre-production alternatives, including service-level fault injection testing [19] and microservice dependency localization via PageRank (MicroRank [20]), provide lighter options before deployment. Benchmark systems with cataloged, replayable faults, such as Train-Ticket [30], make observed failure impact measurable on a running deployment, which is the validation this study does not yet have (Section 7.4). Architecture-based reliability prediction has a long history. Yacoub and Ammar’s methodology for architecture-level reliability risk analysis [31] pioneered combining component dependency graphs with complexity and failure severity to rank components by risk. Cheung’s absorbing Markov chain [32] and the state-, path- and additive models surveyed by Goseva-Popstojanova and Trivedi [33] and Immonen and Niemelä [34] are classic examples, as are the Palladio Component Model [35], layered queueing networks [36] and the AADL Error Model Annex [37]. These approaches answer broader questions but need operational profiles and failure rates that are unavailable at commit time. SaG asks a narrower question from manifests alone: whose failure reaches furthest in the declared topology?

Telemetry-driven methods diagnose faults in running microservices. Examples are Seer [38] and Sage [39], MicroRCA [40], TraceRCA [41], and GNN-based DeepTraLog [42], Eadro [43] and MicroCause [44] (reviewed in [45]). In synchronous microservices, cascades stem from thread-pool exhaustion, RPC timeouts and retry storms [46]. In pub-sub systems they spread through queue saturation and message starvation. All of these methods need traces or metrics from a live system; SaG works before any code runs. SaG’s question is also that of change impact analysis [47] and of dependency management in microservice fleets [48]. Ranking by impact relates to learning-to-rank defect prediction [49], which optimizes the ranking measure directly, as our listwise loss does (Section 4.2).

2.2. Static Code Analysis and Static System Analysis

Static code analysis (SCA) tools such as SonarQube [16] measure complexity [50], cohesion and coupling [17, 51] within individual services to flag defect-prone modules [52, 53, 54, 55]. Comparing sophisticated graph measures against simple structural metrics has a rich history in software engineering defect prediction: Zimmermann and Nagappan [56] evaluated network analysis on dependency graphs, while Premraj and Herzig’s replication study [57] demonstrated that simple code and coupling metrics largely matched network measures. SCA cannot see inter-service messaging, broker saturation or cross-host propagation. Architecture recovery tools reconstruct system-level structure from code [58, 59], and HAROS [60] checks ROS systems statically before launch. SaG’s static system analysis (SSA) uses the declared topology instead, propagating

code-level metrics across architectural dependencies. Architecture-level coupling metrics count a service’s consumers, as afferent coupling [26], the Absolute Importance of the Service [27] and service fan-in [61]. In publish–subscribe systems these counts are not a single edge lookup, because consumers reach producers only through topics, brokers and shared libraries. They are a typed two-hop query (publisher $\rightarrow$ topic $\leftarrow$ subscriber), which SaG’s dependency rules state explicitly; Because this count is exactly the first propagation wave of the reachability oracle used here, this paper reports it as a reference that restates the oracle, not as a predictor (Section 4.4). This lets teams find structural anti-patterns [62, 63] and architectural technical debt [64] in CI/CD [65, 66].

2.3. Multi-Criteria Weighting

Aggregating QoS policies or failure effects into one auditable weight is a multi-criteria decision problem, for which the Analytic Hierarchy Process (AHP) [67] is standard; SaG uses it for the topic weights of Section 3.2 and the composite oracle of Section 4.3. AHP’s consistency ratio detects inconsistent judgments but not matrices back-filled from a chosen answer, a distinction this study reports for its weights (Supplementary Section S4).

2.4. Graph Learning and Explainability

Closed-form indices [24, 25, 68, 69], one of which supplies this study’s training-free baseline, and cascade models of network robustness [10, 8, 9] assume homogeneous, usually undirected graphs, so a single untyped score conflates structurally different elements such as topics, libraries and hosts. Learned node-importance methods, including FINDER [70], DrBC [71] and PowerGraph [72], share the homogeneity assumption, as does learned approximation of betweenness and closeness [73]. Homogeneous GNNs (GCN [74], GraphSAGE [75], GAT [76]) discard relation identity unless it is supplied as a feature. Softmax attention normalizes over neighbors and so computes a weighted mean that cannot count them, whereas sum aggregation can [77, 78], and counting larger substructures is beyond many message-passing architectures [79]; since the strongest reference here is a neighbor count, this distinction matters (Section 6.2). Heterogeneous GNNs (RGCN [80], HAN [81], HGT [82], MAGNN [83]) learn relation-specific transformations. We evaluate both families under matched capacity (Section 6.2). Khodabandeh et al. [84] apply graph attention to microservice call graphs to predict future interactions. We instead predict the impact of removing a node from a declared topology. GNN explainers such as GNNExplainer [85] and PGExplainer [86] explain models in terms of their internal features. SaG’s proposed explanation layer (Supplementary Section S24) instead names ISO/IEC 25010 quality sub-characteristics [87] and a remediation class for each flagged component; it is not evaluated here.

This study belongs with work showing that simple baselines are often competitive with sophisticated learners [88, 89, 90, 91], and, in software engineering specifically, with the finding that network measures on dependency graphs [56] add little over simple coupling metrics once those are measured [57]. Its contribution is of that kind: an evaluation of learning on derived dependency graphs for a pre-deployment ranking task, with a largely negative result where the oracle is cheap and its strongest simple restatement is a known architecture metric, and a positive one for learned surrogates where the oracle is expensive.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 presents the end-to-end architecture of the SaG framework. The shared front end processes Architecture-as-Code manifests, constructs a typed multigraph, projects implicit runtime interactions throughout a logical dependency layer, and extracts typed node properties. These features feed the ranking engines (Section 4) and, separately and with no shared parameters, the proposed explanation layer (Supplementary Section S24).

3.1. Multigraph Definition

A distributed system is described as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

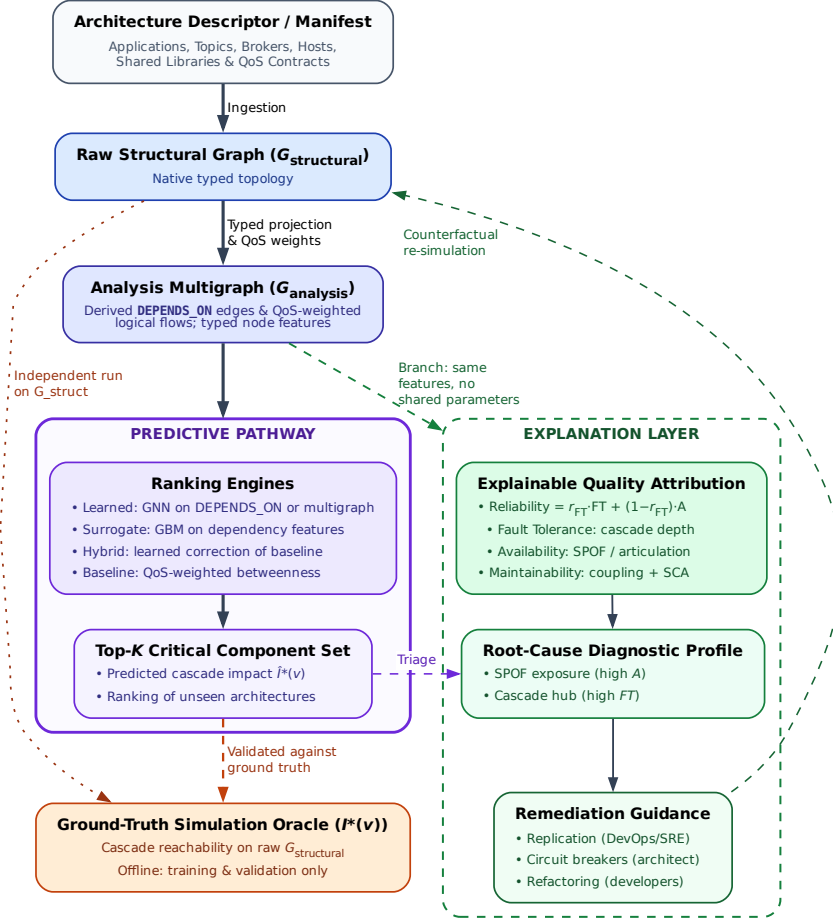


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway runs down the center: manifest ingestion, typed multigraph, `DEPENDS_ON` projection with typed node properties, the ranking engines (training-free baseline, learned and hybrid; Figure 3), and the ranked critical set. The dashed edge marks the ground-truth simulation oracles, which operate only on $G_{\text{structural}}$, train the predictor offline and take no part in inference. The proposed explanation layer (Supplementary Section S24, not evaluated) reads the same analysis multigraph, shares no parameters with the predictor, and is applied to components after they have been ranked; no output of the predictor flows into it.

- $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$ holds the five entity types $\mathcal{T}_V$ of Table 1; V_{host} denotes physical or virtual *Execution Hosts*.
- E is the set of directed edges, and $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ assign entity and relation types.
- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ weight entity criticality and connection strength. For Applications and Libraries, $w_V(v) = 1 - \text{CQP}(v)$, where CQP is a code-quality penalty computed from static code metrics (lines of code, cyclomatic complexity, coupling and cohesion); otherwise $w_V(v) = 1.0$.

3.2. Quality-of-Service Link Weighting

A link's strength depends on its Quality-of-Service (QoS) contract: a `RELIABLE` topic with `TRANSIENT_LOCAL` durability couples services more strongly than a `BEST_EFFORT` telemetry stream. Each topic t carries an aggregate weight $w(t) \in (0, 1]$ combining declared QoS policies (reliability, durability, priority) with payload size and publication frequency. The sub-weights of reliability, durability and priority come from an Analytic Hierarchy Process (AHP) pairwise-comparison matrix that was stated independently rather than back-solved from a target vector ($CR = 0.016$, non-degenerate; Supplementary Section S4, which also shows that three of the framework's other AHP matrices do encode a declared vector).

Table 1: Entity types and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Execution Host (V_{host})	Physical or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO / SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component publishes to / consumes from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on host
CONNECTS_TO	Host $\rightarrow$ Host	Network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library

Table 2: Notation used throughout the paper.

Symbol	Description
$G_{\text{structural}}$	Raw multigraph (used exclusively by simulation oracles)
G_{analysis}	Logical DEPENDS_ON projection (input to predictors)
V_{app}	Application nodes (the primary scored population)
$w(t), w(e)$	QoS topic weight; derived dependency edge weight ($w(e) = w_E(e)$ on DEPENDS_ON)
$I^*(v)$	Primary cascade-reachability simulation oracle
$I_{\text{dyn}}(v)$	Dynamic queue-flow discrete-event simulation oracle
$I_{\text{comp}}(v)$	Multi-criteria composite simulation oracle
ρ	Spearman rank correlation coefficient (full population)
$\rho_{>0}$	Spearman rank correlation restricted to active stratum ($I > 0$)
Overlap@ K	Top- K identification set overlap ($K = 0.20 V_{\text{app}} $)

Whether these weights help a learned ranker is measured directly: with the weighted in-degree held in both arms, the QoS inputs add nothing measurable to the graph neural networks on the reachability oracle (Section 6.2), whereas on the queue-flow oracle they carry the learned surrogate’s gain (Section 6.1). How the weighting affects the training-free baseline is reported in Supplementary Section S37. The reference dependency counts reported in this paper are unweighted and do not read $w(t)$.

3.3. Logical Dependency Projection (**DEPENDS_ON**)

Structural edges do not directly reflect failure propagation: a subscriber depends on a publisher, yet no direct edge joins them in pub-sub topologies. SaG derives an explicit semantic relation, **DEPENDS_ON**, directed from *dependent* to *dependency* (“if the target fails, the source is impacted”), via the rules of Table 3.

Table 3: The **DEPENDS_ON** logical dependency projection rules. [†]Defined for completeness; not exercised by the Application-level evaluation.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic)	$1 - \prod_{t \in T} (1 - w(t))$
2 [†]	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3 [†]	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_E(u \rightarrow v)$
4 [†]	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_E(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6 [†]	broker_to_broker	Broker $\leftrightarrow$ Broker (shared fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine topics T joining a pair by probabilistic union [92, 93, 94], ensuring parallel failure paths increase coupling. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [95], and Rules 3 and 4 lift dependencies to hosts by maximum.

Sequential cascades and simultaneous blasts. Rule 1 captures sequential cascades, where a failed publisher starves subscribers through queues and buffers. Rule 5 captures simultaneous blasts, where a crashed library takes down all dependent applications at once. Libraries that publish or subscribe are endpoints of Rule 1 in their own right, and an Application reaches a library’s topics transitively through Rule 5. The projection on which every count, **Topo-QoS** and every **-P** learner is computed applies Rule 1

to direct subscriptions only; the repository derivation that produces the node features additionally follows up to three **USES** hops, so an Application subscribing through a library it uses is a Rule 1 dependent there. Rules 2, 3, 4 and 6 represent infrastructural and broker dependencies (Table 3).

Remark 1 (the dependency count is a typed two-hop count). Let G_{flow} be the Application–Library projection (Rules 1 and 5) and v an Application. By construction,

$$\text{InDeg}(v) = |\{u \neq v : \exists t \in V_{\text{topic}}, (v, t) \in \text{PUBLISHES_TO} \wedge (u, t) \in \text{SUBSCRIBES_TO}\}|. \quad (2)$$

Since Rule 5 edges end at Libraries, every edge into an Application is a Rule 1 edge $u \rightarrow v$, which exists exactly when $u \neq v$ (an Application or a Library) directly subscribes to a topic that v publishes; parallel topics collapse into one edge.

The right-hand side of Eq. (2) is publish–subscribe afferent coupling (fan-in, the Absolute Importance of the Service [26, 27]), and it is a typed two-hop query on the raw multigraph. **InDeg** therefore needs no projection to compute; the projection states which typed query to ask. The equality is also checked on all seventeen corpus graphs (maximum absolute difference 0). What the projection adds beyond this query is measured separately: Rule 5 raises transitive reach by +0.058 (Section 6.1). Because the primary oracle’s first propagation wave is exactly this set (Section 4.4), **InDeg** and its transitive counterpart **Reach** are used in this paper as references that restate the oracle, not as predictors (Amendment 13).

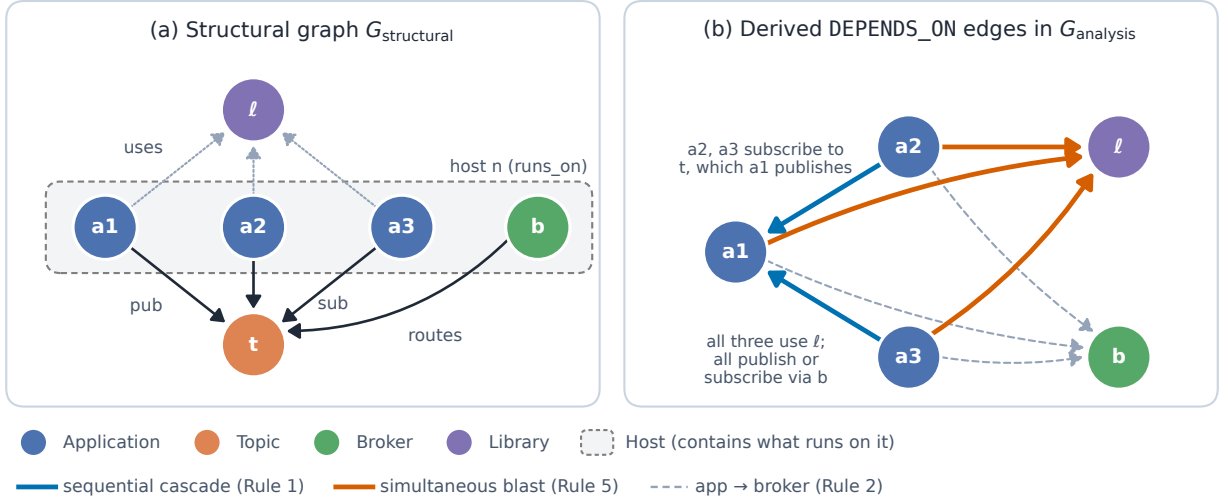


Figure 2: Running example. (a) Three applications share topic t (routed by broker b) and library ℓ , and all run on host n . No structural edge joins two applications. (b) The derived **DEPENDS_ON** edges make the hidden dependencies explicit: subscribers a_2, a_3 depend on publisher a_1 (Rule 1), all applications depend on ℓ (Rule 5), and each application depends on the broker (Rule 2). Simulation oracles run on view (a); predictors read view (b).

3.4. Dual Graph Views

The **structural graph** $G_{\text{structural}}$ is the raw deployment topology. The **analysis graph** G_{analysis} adds the derived **DEPENDS_ON** edges and code metrics (Figure 2). Predictor features are computed on G_{analysis} , while simulation oracles run strictly on $G_{\text{structural}}$ (Section 4.4). The two views also differ for a learner: on $G_{\text{structural}}$ every relation points away from Applications, so forward message passing never reaches them, whereas on the **DEPENDS_ON** graph each Application receives messages from its dependents (Section 6.2).

3.5. Typed Node Feature Encoding

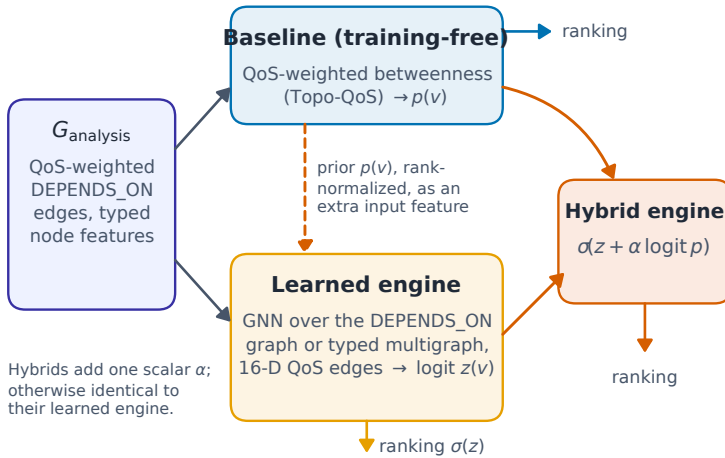
Both the predictive pathway (Section 4) and the proposed explanation layer (Supplementary Section S24) read typed node properties from G_{analysis} . All five entity types share an 18-dimensional base block of normalized topological metrics ($[0, 1]$): PageRank, Reverse PageRank, betweenness, closeness, eigenvector

centrality, in- and out-degree, clustering, articulation, bridge ratio, the node QoS weight, incoming and outgoing dependency weights, multi-path coupling, path complexity, fan-out criticality, and the Connectivity Degradation Index (CDI). Four centrality features (PageRank, Reverse PageRank, betweenness and closeness) are computed over weighted edges and so carry QoS information; the nominal QoS-off condition is therefore not completely QoS-free. The in-degree feature, and the incoming dependency weight w_{in} , a QoS-weighted version of it, mean that every learner is given a quantity closely related to **InDeg** as an input, so the learners inherit part of the reference counts' overlap with the oracle through their features (Section 6.2); a learner without both columns was not run. Type-specific blocks extend the vector to 19–25 dimensions (full schema in Supplementary Section S11). CDI evaluates structural connectivity loss via a fixed-size breadth-first sample to control computation time (Section 6.4); its sensitivity to hash-seeded tie breaking is examined in Section 7.3.

4. Ranking Engines and Ground Truth

SaG ranks components with learned engines, which are compared against one training-free baseline. The **learned engines** are graph neural networks over the typed multigraph and the derived dependency graph (this section), and gradient-boosted surrogates over per-Application dependency-graph features (Section 5.2). The **hybrid engines** are graph neural networks that correct the baseline score (Section 6.1). The **training-free baseline** **Topo-QoS** is QoS-weighted betweenness on the dependency projection (Section 5.2). All are trained or scored against simulation oracles that run on a separate graph view (Sections 4.3–4.4). Figure 3 shows how the engines relate and how they are evaluated. Full hyperparameters and training commands are on the experiment pages of the replication repository (Section 5.1).

(a) Learned engines and baseline, one analysis graph



(b) Ground truth and evaluation

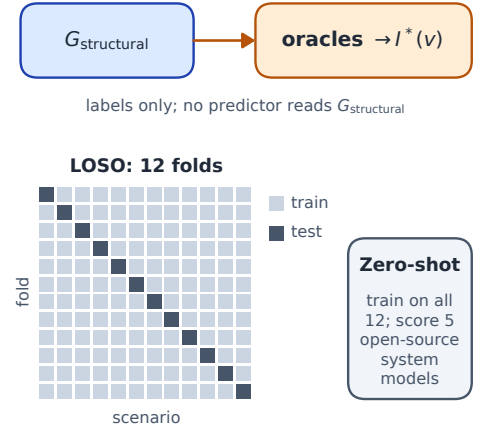


Figure 3: (a) SaG’s ranking engines read the analysis graph. The training-free baseline scores QoS-weighted betweenness; $p(v)$ is that **Topo-QoS** score, rank-normalized to $[0, 1]$ within the graph; the learned engine outputs a logit $z(v)$. A hybrid engine gives the learned engine $p(v)$ as an extra input feature and adds a learned correction to it on the logit scale, $\sigma(z + \alpha \logit p)$, with one learnable scalar α . (b) Ground truth comes from simulation oracles on the structural graph, which no predictor reads. Engines are evaluated by leave-one-scenario-out cross-validation over twelve synthetic architectures and zero-shot on five open-source system models.

4.1. Heterogeneous Graph Transformer and Attention Networks

The primary learned engine **HGT-QoS** is a three-layer Heterogeneous Graph Transformer (HGT) [82] in PyTorch Geometric [96], with hidden dimension $D = 64$ and $H = 4$ heads. Entity-specific linear projections map raw node features $x_v \in \mathbb{R}^{19-25}$ into hidden space, $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. For each

meta-relation $\langle \tau(u), \phi(e), \tau(v) \rangle$, attention uses type-parameterized keys, queries, and values, scaled by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$ (full layer-wise formulations in Supplementary Section S1.1). Message passing runs over both G_{analysis} and its transpose. On the raw multigraph, native relations point away from Applications, so forward message passing cannot reach scored nodes, reducing forward GNNs to per-node MLPs over precomputed centralities (Section 7.2). The homogeneous Graph Attention Network baseline (GAT-QoS, and GAT-P-QoS on the dependency projection) uses a 3-layer architecture with 4 attention heads and width $D = 288$ ($D = 296$ in the unweighted control to match parameter capacity), projecting all node types into a shared embedding space before homogeneous GATConv layers.

Each directed edge carries a 16-dimensional vector $e_{uv} \in \mathbb{R}^{16}$: index 0 represents the coupling weight $w_E(e)$, index 1 the normalized path count, indices 2–8 a relation one-hot, and indices 9–15 middleware QoS parameters (reliability, durability, priority, depart-mode flag, deadline pair, max-blocking time). The edge vector is projected and added before attention, $\tilde{h}_v = h_v + W_{\text{edge}}e_{uv}$.

4.2. Prediction Head and Training Objective

A composite head predicts simulated cascade impact, $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{a}_1(v) \parallel \hat{a}_2(v)))$, where the auxiliary heads $\hat{a}_1, \hat{a}_2$ are a reliability and a maintainability head. $\hat{a}_1$ is regressed on the oracle’s reliability column I_R^* ; the oracle records no maintainability, so $\hat{a}_2$ ’s regression term is masked out and it is trained only through the composite head. The objective combines regression with listwise and pairwise ranking:

$$\mathcal{L} = \text{MSE}(\hat{I}^*, I^*) + 0.5 \cdot \text{MSE}(\hat{a}_1, I_R^*) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}}, \quad (3)$$

where $\mathcal{L}_{\text{rank}}$ is ListMLE [97] over the ground-truth permutation π ,

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\hat{s}_{\pi_i} - \log \sum_{j=i}^N \exp(\hat{s}_{\pi_j}) \right), \quad (4)$$

and $\mathcal{L}_{\text{pairwise}}$ is a margin-ranking loss ($\gamma = 0.05$) over pairs differing by more than γ . Tied labels (about 31% of Applications have $I^* = 0$) enter π in a fixed but arbitrary order, so ListMLE is only approximately defined on them; the pairwise term ignores pairs within γ (Supplementary Section S20).

Models are trained with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4}) under cosine warm restarts for up to 300 epochs, with early stopping at patience 60 on a 20% node-level validation split of the largest training scenario, over five fixed seeds {42, 123, 456, 789, 2024}. Hyperparameters and loss coefficients were fixed before the first sweep and are identical across arms. **This deviates from the registered plan**, whose selection rule chooses every hyperparameter by nested inner-LOSO selection (two-fold inner holdout, Amendment 1); the published learned results use one fixed configuration instead, adopted during primary campaigns to maintain computational tractability across seeds and folds, and no amendment recorded the change (Section 5.3). The registered selection was run afterwards, on the eight-configuration stage-1 grid (depth and rank normalization of features and labels), as a sensitivity analysis; its harness also early-stops on a held-out training scenario, so it differs from the published protocol in that respect too (Section 6.2), confirming that the primary null result remains unchanged under nested tuning.

4.3. Ground-Truth Simulation Oracles

Ground truth is evaluated using simulation oracles on the raw structural multigraph $G_{\text{structural}}$:

- **Primary reachability cascade oracle (I^*):** Crashes component v , propagates outages through dependent topics, brokers and links via breadth-first traversal on $G_{\text{structural}}$, and computes the mean fractional feed loss across intact subscribers. Feed loss is scaled by a QoS severity ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high priority, $\times 1.05$ medium) and clamped to $[0, 1]$. Each propagation wave is processed in a seeded random order, which decides ties at the propagation threshold; the label is the mean over five seeds. Disabling QoS scaling leaves the Application ordering largely intact ($\rho = 0.965$ across the twelve folds), and substituting durability-aware rescaling moves it less still ($\rho = 0.977$; Supplementary Section S9), showing that I^* is predominantly a topological reachability metric.

- **Independent queue-flow discrete-event oracle (I_{dyn}):** A discrete-event SimPy [98] simulation modeling dynamic message emission rates, queue buffer saturation, and network latency. It records the drop in delivered message rate suffered by surviving consumers under active load, and it reads the declared rates, payload sizes and QoS contracts that I^* ignores. Its mechanism is therefore different from reachability, but its substrate is the same pub-sub topology, and its first-order effect—consumers that lose a failed publisher’s messages—is again subscriber loss. It is labeled for all 1,321 Applications, as the mean over five seeds (Amendment 11), at a cost of 12.7 CPU-hours for the corpus against seconds for I^* ; an earlier revision used only the first 30 Applications per fold in lexicographic order, which is kept as a sensitivity check (Supplementary Section S36). I_{dyn} agrees with I^* at mean $\rho = 0.711$ [0.625, 0.794], and its published test–retest reliability is 0.74–0.97 (Supplementary Section S9), which bounds the correlation any deterministic ranker can attain with it.
- **Composite multi-criteria failure oracle (I_{comp}):** An auxiliary benchmark evaluated with the Validate-stage failure simulator, run exhaustively over all 1,321 Applications with QoS weighting on and a primed baseline flow. It scores each removal as $I_{\text{comp}}(v) = 0.35 \text{RL}(v) + 0.25 \text{FR}(v) + 0.25 \text{TL}(v) + 0.15 \text{FD}(v)$: reachability loss, fragmentation (0.70 structural component loss + 0.30 severity), throughput loss and flow disruption. The weights come from one of the three AHP matrices that encode a declared vector rather than elicited judgment (Supplementary Section S4), so I_{comp} serves as a declared multi-objective stress test specifically assessing resilience to topological fragmentation and throughput disruption rather than pure subscriber loss. Because fragmentation and flow disruption respond to almost every removal, $I_{\text{comp}}(v) > 0$ for virtually every Application. I_{comp} is never a training label.

4.4. Input–Label Independence and Construction Bounds

Features are extracted strictly from G_{analysis} , while simulation oracles execute on $G_{\text{structural}}$, and no simulation output is exposed as an input feature; a regression test in the replication package enforces this.

Procedural separation does not remove construct overlap, and this is the paper’s central threat, stated once here. **Criterion.** A ranking is a *reference* for an oracle O if it is a closed-form truncation of O ’s own computation from the same inputs; it then measures how much of O is its own rule, not predictive skill, and no contrast against it is a claim. For I^* , whose first propagation wave is exactly the set **InDeg** counts (Remark 1), the references are its first-order expansion (Eq. (6)), **InDeg**, **Reach** and their raw-graph equivalents (Amendment 13). For I_{dyn} no ranking here truncates its computation, but its first-order effect is the same subscriber loss, so the same rankings are read as references on it too. For I_{comp} no ranking is a truncation either; its fragmentation and flow terms, however, respond to removing high-degree and high-betweenness nodes, so **Topo-QoS** and raw degree are *proximate* to it, and their lead on I_{comp} is read with the same caution as the references’ lead on I^* . The learned engines are not references by this criterion, but they read in-degree features and are trained on I^* , so they inherit part of the overlap (Section 6.2 measures how much). Two analyses bound the circularity rather than remove it: every ranker is scored on all three oracles, and its agreement with I_{dyn} is also reported after the rank of I^* (and, separately, of the first-order term) has been partialled out (Section 6.1). Neither replaces validation against observed failures on physical, running distributed systems, which this study does not have (Section 7.3); all findings are therefore strictly conditional on simulation fidelity.

5. Experimental Setup

5.1. Corpus and Replication Package

The corpus comprises 2,812 components across seventeen architectures (Table 4). Twelve synthetic topologies form the LOSO folds. They span autonomous vehicles, financial trading, healthcare, industrial SCADA, smart-city IoT, telecom RAN, logistics, gaming, microservices, enterprise integration and air-traffic management. All twelve come from one generator family, as do their code metrics, so LOSO measures transfer across parametric configurations of that generator rather than across structurally distinct industrial ecosystems. Each regenerates byte-identically from a committed configuration, and CI verifies this against a SHA-256 manifest.

Table 4: Evaluation corpus. The twelve synthetic topologies are the LOSO folds; the five open-source system models are excluded from all training and used only for zero-shot transfer (Section 6.3). Counts are read from the committed topology files and verified in CI; per-scenario composition: Supplementary Section S13.

Dataset	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

The five open-source systems are hand-authored models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two meshes inspired by Online Boutique and Train-Ticket were each authored by one author as typed multigraphs from public documentation; they are not mechanical extractions. Brokers, QoS profiles, code metrics and host specifications are partly assumed. Two models depart materially from their originals: the Online Boutique model is a 22-application pub-sub mesh with four brokers, whereas the original is an RPC-based service mesh with no broker, and the Train-Ticket model represents its service-discovery server as a broker. Neither contains synchronous call edges. RQ3 therefore tests zero-shot transfer to stylized architecture models inspired by open-source systems created by a single modeler, not to deployed production systems or mechanically extracted call graphs.

Replication package. Datasets, harnesses, checkpoints and result artifacts are archived on Zenodo (see Data Availability). The public repository documents each experiment: its protocol, hyperparameters, `make` target, artifacts and the supplementary section holding its extended results (<https://github.com/onuralpyigit/software-as-a-graph/tree/main/docs/research/jss/experiments>).

5.2. Predictors

Table 5 lists the predictors: the training-free baseline (Topo-QoS), learned engines on the dependency graph and on the raw multigraph, and hybrid engines. Additional training-free baselines (Degree-raw, PR-raw, RevPR-raw, Topo) are reported in Supplementary Section S37. Dependency counts are not in the table: they restate the primary oracle’s propagation rule and are reported as reference rankings (below). On a GNN, `-QoS` means the 16-D QoS edge vector (Section 4.1) together with three QoS node columns (w , w_{in} , w_{out}), and `Hybrid-X` is engine X corrected by the baseline prior. `GAT` and `GAT-QoS` are untyped GATs matched to HGT in parameter budget, and `GAT-QoS` reads the same 16-D edge vector as `HGT-QoS`, so the four learned models form a 2×2 over typing (GAT vs. HGT) and the QoS channel. Smaller and projection-based variants that the study also ran are listed in Supplementary Section S30. The learned and hybrid engines ingest the native typed multigraph under LOSO, and the `-P` learners the Application–Library `DEPENDS_ON` graph, with bit-identical node features and labels. In QoS-off arms every edge weight is 1 and the QoS node columns are zeroed, though four centralities still carry QoS (Section 3.5, Supplementary Section S11).

Training-free baseline. Topo-QoS is computed on the Application–Library `DEPENDS_ON` graph (Rules 1 and 5):

$$\text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v), \quad (5)$$

where BT_w is betweenness over edge distances $d(e) = 1/(w(e) + 10^{-6})$, so strongly coupled edges attract shortest paths, and `AP` flags articulation points. Because of an implementation defect, the articulation term reads zero for every node, so the reported score ranks as QoS-weighted betweenness. Restoring the term lowers `Topo-QoS` from 0.553 to 0.533, so the defect does not favor the reference counts (Supplementary Section S22). The unweighted Topo baseline is defined in Supplementary Section S37.

Reference rankings. Three primary training-free rankings restate the reachability oracle’s propagation rule and are reported as references (Amendment 13): the first-order expansion of I^* (Analytic I^* , Eq. (6)); $\text{InDeg}(v)$, the number of direct dependents of v on the Application–Library `DEPENDS_ON` graph, which for an Application equals publish–subscribe afferent coupling and is exactly the set I^* ’s first wave reaches (Remark 1); and $\text{Reach}(v)$, the number of its transitive dependents normalized by $|V| - 1$, which is the set a reachability cascade can visit. Raw-multigraph topic counting (Pubs-raw) and transitive closure without

Table 5: Predictors reported in the main text. -QoS: QoS-weighted distances (Topo-QoS) or the 16-D QoS edge vector plus three QoS node columns (GNNs); -P: trained on the dependency graph; Hybrid-X: engine X corrected by the Topo-QoS prior. ¶Topo-QoS carries an implementation defect where the articulation point term is zero (corrected score is 0.533). Reference rankings that restate the oracle (Analytic I^* , InDeg, Reach) and GAT-P+InDeg, which uses one as its prior, are not predictors (Amendment 13). Other training-free baselines and exploratory raw-graph references (Pubs-raw, Reach-R1) are reported in Supplementary Section S37. Each dependency-graph learner keeps its raw-multigraph counterpart’s architecture and parameter budget. All runs, with substrate details: Supplementary Section S30.

Predictor	Graph read	Typing / edge input	Param
<i>Training-free baseline (registered comparator)</i>			
Topo-QoS [¶]	App-Lib DEPENDS_ON	scalar $w(e)$; betweenness only (AP term zero; corr. 0.533)	
<i>Learned, dependency graph (Amendment 9)</i>			
GAT-P-QoS, HGT-P-QoS	App-Lib DEPENDS_ON	homogeneous / heterogeneous	429,992 / 434,62
<i>Learned, raw multigraph: typing $\times$ QoS channel at matched budget</i>			
GAT / GAT-QoS	$G_{\text{structural}}$ + features	homogeneous; none / 16-D	437,496 / 429,99
HGT / HGT-QoS	$G_{\text{structural}}$ + features	heterogeneous; 1-hot / 16-D	434,62
Hybrid-GAT / Hybrid-HGT	$G_{\text{structural}}$ + features	as base + Topo-QoS prior	431,433 / 434,94

Rule 5 (Reach-R1) are reported in Supplementary Section S37. A reference shows how much of an oracle a restatement of its rule recovers, which is the size of the circularity, not predictive skill. References carry no contrast against Topo-QoS, and a learner’s distance from them is reported as such, not as a test.

5.3. Metrics, Protocols and Statistics

Population. Every predictor is scored on the Application set V_{app} .

Metrics. Ranking is measured by Spearman ρ against the simulation oracles. To separate rank ordering among active components from the trivial agreement on inert components with zero impact, we report both full-population Spearman ρ and active-stratum Spearman $\rho_{>0}$ (restricting to components with true impact > 0). Critical-set identification is measured by $\text{Overlap}@K$, the fraction of the true top- K recovered by the predicted top- K with $K = \text{round}(0.20 |V_{\text{app}}|)$. PR-AUC, $F_1@7$ and nDCG@10 are reported in Supplementary Section S15.

Protocols. Under *LOSO*, learned models train on eleven scenarios and are tested on the twelfth, over all 12 folds and five seeds; the reported value is the mean over seeds of each seed’s ρ . Under *zero-shot transfer*, models trained on all twelve scenarios are evaluated without fine-tuning on the five system models. Training-free rankers are fitted to nothing, so for them both protocols reduce to *per-scenario evaluation* on the same twelve scenarios and five models; they are placed in the same tables only so that every row is scored on identical scenarios and labels.

Statistics. Summary ρ is the arithmetic mean of the per-fold (or per-system) Spearman correlations, with a 95% CI from a bootstrap that resamples folds ($B = 2,000$) [99]; comparisons use paired two-sided Wilcoxon signed-rank tests over folds [100]. With 12 folds, the smallest attainable two-sided p is 0.00049. Fisher- z averaging leaves the order of every row of Table 6 unchanged. Weighting folds by $|V_{\text{app}}|$ leaves the top five unchanged (the reference InDeg rises to 0.810) but lifts Topo-QoS to 0.596, level with the raw-multigraph learned engines (0.595–0.606) (Supplementary Section S35). Spearman ρ assigns tied scores their average rank. $\text{Overlap}@K$ breaks ties at the K boundary by node order, which is arbitrary for integer counts; Section 6.1 therefore also reports tie-aware recall curves. Because LOSO folds share ten of eleven training scenarios, every registered contrast is also tested with the Nadeau–Bengio corrected resampled t [101] and, under $|V_{\text{app}}|$ weighting, with an exact sign-flip permutation test (Supplementary Section S36).

Analysis plan and the status of each result. The analysis plan and its amendments are a version-controlled file in the replication repository, each entry committed before the results it governs, with dates and commit hashes listed in Supplementary Section S25; they are not deposited with an independent registry, so their timestamps are the repository’s own. One rule assigns every result to one of three tiers. *Confirmatory*: only the original plan (2026-09-06), HGT-QoS vs. Topo-QoS (primary) and HGT vs. Topo-QoS, Holm-corrected across the two. *Registered secondary*: every amendment written before any result of its own arms existed—the matched 2×2 (Amendment 2), the two hybrids (Amendments 5 and 6, 2026-09-23/24), the reference counts

and QoS controls (Amendment 7), the dependency-graph learners (Amendment 9), the value of the derivation (Amendment 10), the full-population I_{dyn} and learned combinations (Amendment 11) and the round-8 arms (Amendment 14), each Holm-corrected within its own family. All of them were written after the primary null was known, and the choice of arms was informed by it. *Exploratory*: analyses registered after their results existed (Amendments 8, 12 where marked, and 13) and every descriptive analysis. Amendment 13 reclassifies the dependency counts as references (Section 4.4); it changes no number. **One deviation from the plan**: its selection rule chooses every hyperparameter by nested inner selection, but the published learned results use one fixed configuration, and no amendment recorded that change; Amendment 14 runs the registered selection as a sensitivity analysis (Section 6.2). As a sensitivity check, the 13 contrasts of the plan and Amendments 2, 5 and 6 are also pooled under one omnibus Holm correction (`reproduce/omnibus_holm.py`): both hybrid contrasts against **Topo-QoS** stay below 0.05 (Hybrid-GAT $p_{\text{omni}} = 0.019$, Hybrid-HGT $p_{\text{omni}} = 0.041$), and the primary contrast does not ($+0.069$, nominal $p = 0.266$, $p_{\text{omni}} \geq 0.46$).

6. Results

All results are reported on the Application population (V_{app}) against the primary oracle $I^*(v)$, and, for the rankers of Table 7, also against the queue-flow oracle $I_{\text{dyn}}(v)$ and the multi-criteria oracle $I_{\text{comp}}(v)$ (Section 4.3). Only the primary and secondary contrasts are confirmatory; every other contrast is registered secondary or exploratory (Section 5.3). Per-fold results, secondary strata and extended protocol notes are in the Supplementary Material and public experiment pages (Section 5.1). Figure 4 summarizes the main findings.

6.1. RQ1: Ranking Accuracy of Learned Engines

Summary. Derived dependency graphs enable highly accurate cascade-impact ranking: the graph attention network (**GAT-P-QoS**) reaches $\rho = 0.748$ against the primary reachability oracle I^* . Causal ablations reveal that it achieves this strong performance by distilling the underlying degree distribution (losing 0.14 without degree features; Section 6.2), matching the first-order reference level (0.764, indistinguishable; not equivalent within ± 0.05). On the expensive queue-flow oracle I_{dyn} (12.7 CPU-hours), a learned surrogate over dependency-graph features achieves $\rho = 0.799$, significantly outperforming every closed-form approximation (0.706, Holm $p = 0.019$) and capturing declared QoS signals ($+0.095$). Under multi-criteria failure conditions (I_{comp}), hybrid engines maintain robust stability (0.58) where pure neural models collapse (0.144–0.334). On the raw multigraph, the registered primary contrast (**HGT-QoS** vs. **Topo-QoS**, 0.553; corrected 0.533) was null ($\Delta\rho = +0.069$, $p = 0.266$). Recovering 80% of the true top-20% set requires flagging the top 40% by **GAT-P-QoS**.

Learned engines are trained on eleven scenarios and scored on the twelfth; training-free rankers are simply scored on each of the twelve (26 to 300 Applications, K between 5 and 60). Table 6 reports all of them against I^* , with the reference rankings of Section 5.2 in a separate block: they restate I^* 's rule, so their rows measure the circularity and carry no contrast.

The reference level. By Remark 1, **InDeg** is publish–subscribe afferent coupling [26, 27, 61] and counts exactly the Applications that I^* 's first propagation wave reaches; **Reach** counts the transitive dependents a reachability cascade can visit. Their values on I^* (0.764, $\rho_{>0} = 0.516$; 0.732) therefore measure how much of the oracle a restatement of its rule recovers, not predictive skill. Exploratory raw-multigraph references are reported in Supplementary Section S37: counting published topics (Pubs-raw) reaches 0.731, and transitive closure without Rule 5 (Reach-R1) 0.674. Among the references, the derivation contributes in one measured place: without Rule 5, **Reach** falls by 0.058 (9/12 folds, Holm $p = 0.0068$; Amendment 10). Untyped raw-graph metrics recover none of the reference: total degree reaches 0.199 and reverse PageRank 0.089, and PageRank is constant for every Application because Applications have no incoming raw edges. **The first-order expansion of the oracle.** The strongest reference is a post hoc closed-form first-order expansion of I^* :

$$\hat{I}_1^*(v) = \sum_{t \in \text{pub}(v)} \frac{|\text{sub}(t)|}{|\text{pub}(t)|}, \quad (6)$$

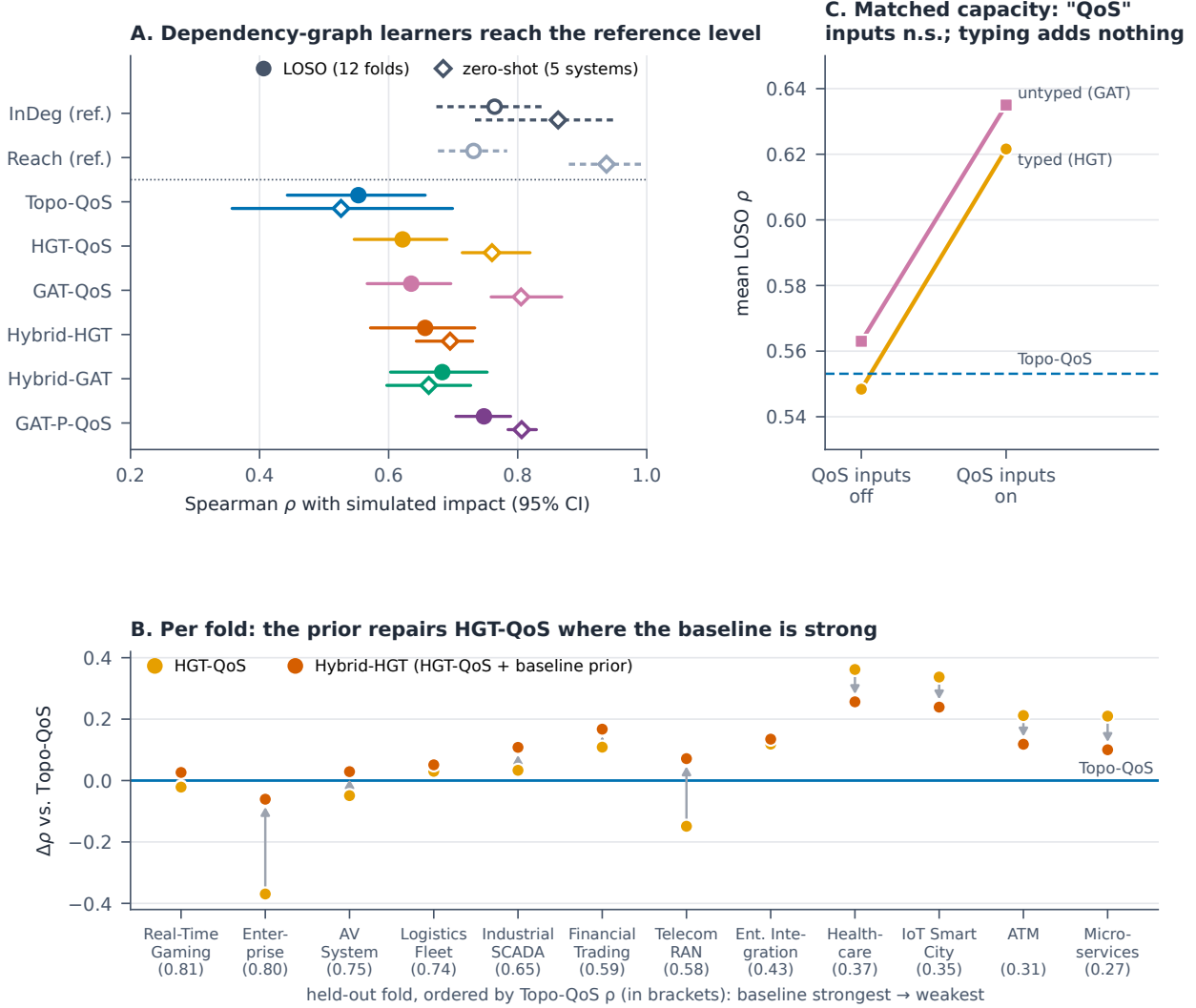


Figure 4: Main results at a glance, Application population. (A) Mean Spearman ρ with 95% bootstrap intervals under LOSO (filled circles; Table 6) and zero-shot on five system models (open diamonds). Grey hollow markers are reference rankings that restate I^* 's propagation rule (**InDeg**, **Reach**; Amendment 13), not predictors; the neural model reading the dependency graph (**GAT-P-QoS**) comes close to **InDeg**; the two are statistically indistinguishable but not equivalent. (B) Per held-out fold, the gain of **HGT-QoS** and of **Hybrid-HGT** over **Topo-QoS**. (C) Cell means of the capacity- and channel-matched 2×2 (Table 8): the "QoS" inputs, which include a QoS-weighted in-degree column, raise both models by about 0.07 (not significant after Holm), while typed and untyped configurations are indistinguishable; with w_{in} held in both arms the "QoS" effect falls to +0.030 (Table 9).

Table 6: Main results under LOSO across twelve synthetic architectures (Application population; learned engines: five seeds, CPU sweeps). $\Delta\rho$ is paired by fold against Topo-QoS, with a bootstrap 95% CI ($B = 2,000$) and a two-sided Wilcoxon signed-rank test. $\rho_{>0}$ denotes Spearman correlation restricted to active components ($I^* > 0$). p_{Holm} is given within each registered family: hybrids (Amendments 5 and 6, registered secondary). The reference block holds canonical rankings that restate I^* 's propagation rule (the post hoc first-order expansion, Eq. (6); the dependency counts of Amendment 7); they are not predictors and carry no contrast (Amendment 13). [‡]Exploratory: Amendment 9 registered its contrasts against the references and raw counterparts. [¶]Topo-QoS carries an implementation defect (articulation term zero; Section 5.2); corrected value is 0.533. Additional training-free baselines and exploratory raw-graph references (Pubs-raw, Reach-R1, Degree-raw, RevPR-raw, Topo) are reported in Supplementary Section S37. Underlined: best predictor per column. Per-fold values, contrasts among references, and GAT-P+InDeg, whose prior is a reference: Supplementary Sections S23, S33, S34 and S35.

Predictor	LOSO ρ [95% CI]	Active $\rho_{>0}$	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@K
<i>Reference: restatements of I^*'s propagation rule (not predictors; Amendment 13)</i>						
Analytic I^*	0.808 [0.755, 0.853]	0.631	—	—	—	0.536
InDeg (direct dependents)	0.764 [0.674, 0.840]	0.516	—	—	—	0.504
Reach (transitive dependents)	0.732 [0.674, 0.782]	0.286	—	—	—	0.344
<i>Training-free baseline (registered comparator)</i>						
Topo-QoS [¶]	0.553 [0.443, 0.657]	0.280	—	—	—	0.388
<i>Learned, on the dependency graph (Amendment 9)</i>						
GAT-P-QoS	<u>0.748</u> [0.704, 0.789]	<u>0.440</u>	+0.195 [+0.100, +0.294]	10/12	0.0068 [‡]	<u>0.454</u>
HGT-P-QoS	0.514 [0.392, 0.636]	0.237	−0.039 [−0.155, +0.077]	4/12	0.622 [‡]	0.380
<i>Learned and hybrid, on the raw multigraph</i>						
HGT-QoS	0.622 [0.547, 0.690]	0.312	+0.069 [−0.046, +0.174]	8/12	0.266	0.426
GAT-QoS	0.635 [0.567, 0.696]	0.338	+0.082 [−0.046, +0.201]	7/12	0.233	0.438
Hybrid-HGT	0.657 [0.572, 0.733]	0.345	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435
Hybrid-GAT	0.683 [0.603, 0.753]	0.362	+0.130 [+0.075, +0.190]	11/12	0.0015 (0.0029)	0.450

where $|\text{pub}(t)| \geq 1$ holds for all published topics $t \in \text{pub}(v)$ by construction (v publishes to t), preventing any division by zero. It reaches mean $\rho = 0.808$ [0.755, 0.853] ($\rho_{>0} = 0.631$): direct subscriber loss is most of what I^* measures, and InDeg is a coarser version of the same quantity. As a reference it bounds the mean rather than every fold: on four folds (AV, Financial Trading, Healthcare, Industrial SCADA) InDeg slightly exceeds it (by 0.005–0.038).

Learners on the dependency graph approach the reference. The same graph neural networks, trained on the dependency graph instead of the raw multigraph, reach $\rho = 0.748$ (GAT-P-QoS), the best GNN in Table 6, and 0.016 below the direct-dependent count on the per-seed statistic; its five-seed ensemble is 0.007 above it (0.772, Table 7). Neither difference is significant, and neither is equivalence: the 90% interval of the per-fold difference reaches ± 0.076 , so a two one-sided test at the registered margin of ± 0.05 fails ($p = 0.17$ per seed, 0.13 for the ensemble; Amendment 14, F3). The learner and the count are therefore indistinguishable at this sample size, not ranked. The approach to the reference rests on the degree features every learner receives (Section 3.5): removing `in_degree` and w_{in} lowers GAT-P-QoS to 0.612 (Section 6.2). A variant that uses InDeg as its prior is reported only in Supplementary Section S34, because its prior is a reference. HGT-P-QoS reaches only 0.514 and is highly seed-dependent (mean within-fold seed SD 0.254; Section 7.3).

Hybrids outperform the registered comparator, and little else. Hybrid-HGT reaches $\rho = 0.657$ (+0.103, Holm $p = 0.0068$) and Hybrid-GAT 0.683 (+0.130, Holm $p = 0.0029$) against Topo-QoS, each on 11 of 12 folds; both stay significant under the omnibus Holm correction ($p_{\text{omni}} = 0.041$ and 0.019), under the Nadeau–Bengio corrected t ($p = 0.019$ and 0.014) and under $|V_{\text{app}}|$ -weighted averaging (exact sign-flip $p = 0.038$ and 0.005; Supplementary Section S36). They are the study's only significant registered improvements, and they are weaker than they look, because a hybrid receives $p(v)$ as an input and adds a learned correction to its logit, so it nests its comparator. Against their own base learners, which Amendments 5 and 6 registered alongside, neither is significant (Hybrid-HGT vs. HGT-QoS +0.035, $p = 0.73$; Hybrid-GAT vs. GAT-QoS +0.048, $p = 0.30$). The prior also costs transfer: zero-shot, both hybrids fall below their base learners (Section 6.3). On their own, the raw-multigraph learned engines do not differ significantly from Topo-QoS (HGT-QoS +0.069, $p = 0.266$, the null primary contrast; GAT-QoS +0.082, $p = 0.233$).

Beyond the reachability oracle. Table 7 scores the rankers against the two other oracles. On full-population I_{dyn} , which agrees with I^* at $\rho = 0.711$ [0.625, 0.794], the references stay ahead of every GNN (first-order expansion 0.706, InDeg 0.664, against 0.496–0.615), and Topo-QoS reaches 0.471. Because I_{dyn} shares the topology and the first-order effect with I^* , the partial correlation shows how much of each ranking

Table 7: Rankers against three oracles, twelve LOSO folds, Application population (registered secondary and exploratory). All three oracles are exhaustive over the 1,321 Applications; I_{dyn} is the five-seed mean of Amendment 11 (the earlier lexicographic $n = 30$ sample is in Supplementary Section S36). Partial ρ : Spearman correlation with I_{dyn} after the rank of I^* , or of the first-order expansion $\hat{I}_1^*$ (Eq. (6)), is regressed out of both. Published learned rows score the mean of the five seeds’ *predictions*, which raises ρ on I^* above the per-seed means of Table 6; Amendment 11’s GBM rows and Amendment 14’s surrogate are per-seed means. *Trained on I_{dyn} labels of the eleven training scenarios: a surrogate for I_{dyn} , not an independent check of it. Reference rows restate I^* ’s rule (Section 4.4). Underlined: highest value per column among predictors. 95% CIs bootstrap folds.

Ranker	$I^* \rho$	$I_{\text{dyn}} \rho$ [95% CI]	Partial $\rho(\cdot, I_{\text{dyn}} I^*)$ [95% CI]	Partial $ \hat{I}_1^*$	$I_{\text{comp}} \rho$
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>					
Analytic I^*	0.808	0.706 [0.629, 0.783]	0.318 [0.234, 0.412]	—	0.636
InDeg	0.764	0.664 [0.562, 0.758]	0.272 [0.188, 0.366]	0.079	0.650
Reach	0.732	0.583 [0.510, 0.651]	0.117 [0.055, 0.184]	0.102	0.302
<i>Training-free baseline</i>					
Topo-QoS	0.553	0.471 [0.381, 0.559]	0.134 [0.061, 0.204]	−0.091	<u>0.702</u>
<i>Learned GNNs (seed ensemble)</i>					
GAT-P-QoS	0.772	0.615 [0.549, 0.673]	0.111 [0.016, 0.213]	0.182	0.274
Hybrid-GAT	0.702	0.599 [0.514, 0.669]	<u>0.173</u> [0.082, 0.267]	0.056	0.585
Hybrid-HGT	0.672	0.573 [0.497, 0.638]	0.162 [0.074, 0.252]	0.029	0.582
HGT-QoS	0.667	0.549 [0.464, 0.622]	0.130 [0.035, 0.224]	0.212	0.201
GAT-QoS	0.647	0.523 [0.436, 0.606]	0.107 [0.018, 0.199]	<u>0.217</u>	0.144
HGT-P-QoS	0.618	0.496 [0.395, 0.601]	0.112 [0.033, 0.190]	0.087	0.334
<i>Dynamic queue-flow surrogates (Amendment 11 and Amendment 14; trained on I_{dyn})</i>					
GBM-Dep-QoS→dyn*	<u>0.764</u>	<u>0.799</u>	—	—	0.477
GAT-P-QoS→dyn*	0.734	0.598	—	—	0.368

survives outside the oracle it was fitted to or restates. After the rank of I^* is removed, **InDeg** keeps 0.272 [0.188, 0.366], **Reach** 0.117, **Topo-QoS** 0.134 and the GNNs 0.107–0.173, every interval excluding zero. After the first-order expansion is removed instead, the order reverses: **InDeg** keeps 0.079, and the raw-multigraph GNNs keep most (0.212–0.217), so the learners carry queue-flow signal beyond the direct count that the count does not. Learning from the right labels matters more than the graph model. A gradient-boosted model over the counts, the first-order expansion and QoS features, trained on I_{dyn} labels of the training scenarios, ranks held-out architectures at 0.799, above the first-order expansion (+0.094, Holm $p = 0.019$) and **InDeg** (+0.135, Holm $p = 0.019$; Amendment 11, Family B); its QoS features carry that gain (+0.095 over the same model without them, Holm $p = 0.014$), whereas on I^* they add nothing (−0.003, $p = 0.62$; Family C). The GNN given the same labels does not: **GAT-P-QoS→dyn** reaches 0.598, below the first-order expansion (−0.108, Holm $p = 0.014$) and the gradient-boosted surrogate (−0.201, Holm $p = 0.0015$), and no better than the same GNN trained on I^* (−0.008; Amendment 14, F5). Importantly, these two surrogates differ in their input representations as well as their model classes: the tree model was supplied with the analytical first-order expansion, declared publication rates, and message payload sizes alongside topological counts, whereas the GAT surrogate operated solely over topological graph features. While this confirms that gradient-boosted trees over domain-informed features distill queue-flow mechanics effectively, isolating the role of architectural inductive bias versus input feature disparity remains an open question requiring future feature-matched evaluations (Section 7.2). These surrogates are trained on the oracle they are scored on; their value is cost, since labeling I_{dyn} took 12.7 CPU-hours for the corpus (Section 6.4). Trained on I^* instead, the same gradient-boosted model is less robust across the three oracles than the first-order expansion alone (worst-case $\rho = -0.121$, Holm $p = 0.001$; Family A; Supplementary Section S37).

The multi-criteria oracle favors different rankers. On I_{comp} , which adds fragmentation, throughput and flow disruption (Section 4.3), learned engines without a prior collapse (0.144–0.334; the sum-aggregation GNNs of Section 6.2 reach 0.41–0.44), while the two hybrids, which carry **Topo-QoS** as a prior, keep 0.58. Training-free scores proximate to that oracle’s terms rank higher: **Topo-QoS** reaches 0.702 [0.649, 0.753] (above the direct-dependent reference 0.650; reference ahead on only 3 of 12 folds, $p = 0.110$), while raw total degree reaches 0.719 (Supplementary Section S37). **Reach** falls to 0.302. Their lead is read as the same

kind of overlap with I_{comp} 's fragmentation and flow terms as the references' lead on I^* (Section 4.4), not as predictive skill. No ranker is best on all three oracles.

Identifying the critical set. Rank correlation over all Applications is not the operational question in a CI gate, which is whether the true top of the ranking gets flagged. Figure 5 gives, for each ranker, the share of the true top-20% set on I^* that falls within its top $k\%$, with ties resolved in expectation (Amendment 12). At $k = 20\%$, GAT-P-QoS recovers 0.48 of the critical set and Topo-QoS 0.38, consistent with their $\text{Overlap}@K$ (0.454 and 0.388): about half of what a top-20% gate flags is not in the true top 20%, and about half of the true top 20% is missed. Recovering 80% of the critical set requires flagging the top 40% by GAT-P-QoS (0.81), and Topo-QoS does not reach 80% within 50%. The references do little better: at $k = 20\%$, InDeg recovers 0.49 (tie-breaking bounds 0.48–0.50), and 80% needs the top 45% by InDeg (0.83) or by the first-order expansion (0.89); only the first-order expansion reaches 90% within the top 50%, and Reach does not reach 80% within 50%. On the earlier $n = 30$ I_{dyn} developmental sample (the right panel of Figure 5; preserved for tie-breaking recall inspection; Table 7 reports the exhaustive $N = 1,321$ population), GAT-P-QoS reaches 0.65 at 40% and 0.80 at 50%, against 0.83 at 40% for the InDeg reference. In an industrial deployment, requiring a gate to flag 40% to 50% of components to catch 80% of critical points introduces substantial developer alert fatigue, indicating that SaG is most effective as a prioritized triage list for architectural inspection rather than an automated blocking gate.

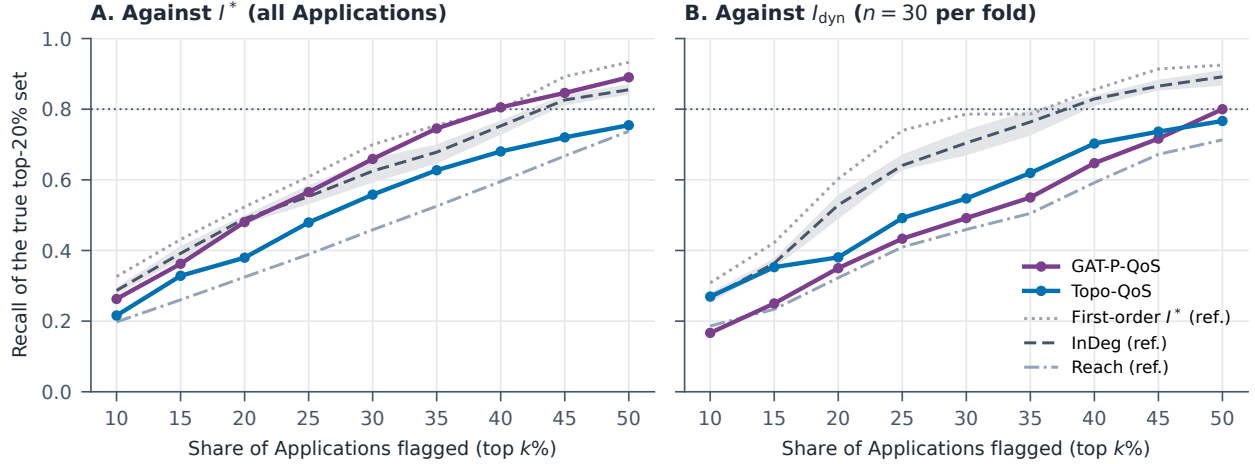


Figure 5: Recall of the true top-20% set (tie-inclusive) by each ranker's top $k\%$, mean over the twelve LOSO folds, against I^* (left) and the exploratory $n = 30$ I_{dyn} developmental sample (right; Table 7 reports the exhaustive $N = 1,321$ population). Ties are resolved in expectation over random tie-breaking; grey dashed curves are reference rankings that restate I^* 's rule (Amendment 13), and the shaded band gives InDeg's optimistic and pessimistic bounds. Reach, whose scores tie heavily, has much wider bounds (0.12–0.80 at $k = 20\%$ on I^*), so its expected curve is the only meaningful summary. The dotted line marks 80% recall.

6.2. RQ2: What Learned Engines Need

Summary. Learned engines gain +0.08 to +0.11 from reading the dependency graph, where messages reach the scored Applications from their dependents (11–12 of 12 folds). What brings the dependency-graph GAT close to the reference is its degree features: without them it loses 0.14 (Holm $p = 0.005$). A sum-aggregation GNN, which can count dependents, loses only 0.01 without them, although the aggregator contrast itself is not significant after correction. With capacity matched, relation typing adds nothing; with the weighted in-degree held in both arms, the “QoS” inputs add nothing measurable either (+0.030, Holm $p = 0.33$).

The “QoS” factor was mostly the weighted in-degree. In Table 8 its main effect (+0.073) is not significant after Holm correction ($p_{\text{Holm}} = 0.127$), and it could not be attributed to QoS content in any case: w_{in} , the sum of incoming dependency weights, is a QoS-weighted version of InDeg, and the QoS-off arms zeroed it. Amendment 14 reran the matched 2×2 with w_{in} kept in the QoS-off cells (Table 9, F4): the main effect falls to +0.030 [−0.023, +0.085] (Holm $p = 0.33$), typing is −0.036 and the interaction +0.046, none significant. More than half of the earlier QoS effect was therefore the weighted in-degree. The closed-form

Table 8: The 2×2 with capacity and edge-channel width matched (Amendment 2): GAT ($\neg T \neg Q$, 437,496 parameters), HGT ($T \neg Q$, 434,620), GAT-QoS ($\neg T Q$, 429,992), HGT-QoS ($T Q$, 434,620). One CPU sweep, twelve LOSO folds, five seeds, Application population. Holm correction across the three orthogonal quantities. Cell means: 0.563, 0.548, 0.635, 0.622.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
<i>Three orthogonal quantities, Holm-corrected across these three</i>							
Typing (main effect)	averaged over Q	-0.014	[-0.052, +0.023]	4/12	29.0	0.470	0.940
“QoS” inputs (main effect)	averaged over T	+0.073	[+0.013, +0.120]	10/12	13.0	0.043	0.127
Typing $\times$ “QoS” interaction	difference of differences	+0.001	[-0.050, +0.042]	6/12	34.0	0.733	0.940
<i>Simple effects — descriptive, not separately corrected</i>							
Typing, QoS absent	HGT vs. GAT	-0.015	[-0.064, +0.033]	5/12	31.0	0.569	—
Typing, QoS present	HGT-QoS vs. GAT-QoS	-0.013	[-0.054, +0.026]	4/12	27.0	0.380	—
“QoS” inputs, typing absent	GAT-QoS vs. GAT	+0.072	[+0.028, +0.109]	10/12	9.0	0.016	—
“QoS” inputs, typing present	HGT-QoS vs. HGT	+0.073	[-0.002, +0.136]	10/12	19.0	0.129	—

controls point the same way: unweighted betweenness (0.591) and constant topic weights (0.595) match or exceed QoS-weighted betweenness (0.553), and permuting QoS profiles across topics changes nothing ($\Delta = -0.006$ [-0.025, +0.015], $p = 0.733$). On I^* we find no evidence that QoS contract content carries ranking signal; on I_{dyn} , whose mechanics read the contracts, it does (Section 6.1, Amendment 11 Family C).

Table 9: Amendment 14 arms: twelve LOSO folds, five seeds, one CPU invocation with every comparator re-run (each reproduces its published per-seed ρ exactly), Application population. $\Delta\rho$ is paired by fold against the comparator named, with a bootstrap 95% CI; Holm within each registered family (F1: degree features; F2: aggregator; F4: the 2×2 with w_{in} held). —deg: **in_degree** and w_{in} zeroed; —deg*: also PageRank, closeness and eigenvector centrality, whose correlation with **InDeg** is 0.74–0.77 on average (Supplementary Section S36). Zero-shot: mean ρ on the five system models. Registered secondary.

Arm	Comparator	LOSO ρ (I^*)	$\Delta\rho$ [95% CI]	Won	p_{Holm}	$I_{\text{dyn}} / I_{\text{comp}}$	Zero-shot
<i>F1: removing the degree features</i>							
GAT-P-QoS—deg	GAT-P-QoS (0.748)	0.612	-0.136 [-0.205, -0.072]	1/12	0.0049	0.485 / 0.156	0.821
GAT-P-QoS—deg*	GAT-P-QoS	0.587	-0.161 [-0.252, -0.080]	1/12	0.0049	0.462 / 0.138	0.654
GAT-QoS—deg	GAT-QoS (0.635)	0.365	-0.270 [-0.344, -0.199]	0/12	0.0015	0.295 / -0.047	0.810
<i>F2: sum aggregation (GINE, 434,123 parameters) instead of softmax attention</i>							
GIN-P-QoS	GAT-P-QoS	0.732	-0.016 [-0.055, +0.017]	7/12	0.910	0.606 / 0.407	0.806
GIN-P-QoS—deg	GAT-P-QoS—deg	0.721	+0.108 [+0.027, +0.197]	8/12	0.157	0.602 / 0.428	0.810
GIN-P-QoS—deg*	GAT-P-QoS—deg*	0.711	+0.124 [+0.031, +0.230]	9/12	0.157	0.589 / 0.438	0.776
<i>F4: matched 2×2 with w_{in} held (cells GAT+w_{in} 0.628, HGT+w_{in} 0.569, GAT-QoS 0.635, HGT-QoS 0.622)</i>							
Typing (main effect)	averaged over Q	—	-0.036 [-0.079, +0.004]	2/12	0.330	—	—
“QoS” inputs (main effect)	averaged over T	—	+0.030 [-0.023, +0.085]	9/12	0.330	—	—
Interaction	difference of differences	—	+0.046 [-0.009, +0.101]	9/12	0.330	—	—

What the dependency-graph learner needs. Every learner receives in-degree and w_{in} as node features, and on the dependency graph it also aggregates exactly the dependent neighborhood, so its closeness to **InDeg** could come from either. Amendment 14 separates the two (Table 9). Removing the two degree columns costs GAT-P-QoS 0.136, and removing the three centralities most correlated with in-degree as well costs 0.161 (F1, rule F1a): the approach to the reference depends on the degree features. That is what message-passing theory predicts for this architecture: softmax attention normalizes over neighbors and computes a weighted mean, which cannot count them, whereas a sum aggregator can [77, 78]. A GINE network of the same parameter budget bears this out descriptively, keeping 0.721 and 0.711 without the same columns, 0.01–0.02 below its own full-feature score, but the aggregator contrasts miss significance after Holm correction (+0.108 and +0.124, $p_{\text{Holm}} = 0.157$; rule F2b), so the study does not claim an aggregator effect. No dependency-graph learner, with or without degree features, is equivalent to **InDeg** within ± 0.05 (equivalence bounds 0.063–0.276; F3). Zero-shot, the degree columns matter little (0.810–0.821 without them), because on the system models most of the ranking is separating inert from active components (Section 6.3).

Relation typing under matched capacity. HGT-QoS remains within 0.013 of GAT-QoS (Table 8). On the raw multigraph this says little about typing, because no message reaches an Application there: a gradient-boosted model over the same per-node features without any graph (GBM-Feat, Amendment 8) reaches 0.642, level with GAT-QoS, and removing HGT’s reverse pass, the only route by which messages reach Applications, changes nothing (HGT-QoS-U, -0.010, $p = 0.91$; Supplementary Section S25). On the dependency graph, where messages do arrive, HGT-P-QoS reaches 0.514 with a mean within-fold seed SD of

0.254, against 0.748 and 0.030 for **GAT-P-QoS**. This is one fixed configuration (width 100), and averaging its five seeds’ predictions raises it to 0.618; we therefore read it as seed instability of that configuration, not as evidence against relational typing in general. **The registered selection rule.** The plan chose every hyperparameter by nested selection, which the published results did not apply (Section 5.3). Run afterwards on the stage-1 grid (Amendment 14, arm N), it moves **HGT-QoS** from 0.622 to 0.677 (+0.055) and **GAT-P-QoS** from 0.748 to 0.693 (−0.054), neither significant (Holm $p = 0.259$); nested **HGT-QoS** against **Topo-QoS** is +0.123 on 9 of 12 folds, still not significant (Holm $p = 0.192$; rule F6b). Applying the registered rule therefore does not change the primary result. Table 6 reports the fixed, published configuration; Arm N is a registered secondary sensitivity analysis of it. The search harness also early-stops on a held-out training scenario rather than on the published node-level split, and on the three folds where it chose the published hyperparameters it does not reproduce the published scores (up to 0.10 apart), so these differences cannot be attributed to hyperparameter selection alone (Supplementary Section S36).

6.3. RQ3: Zero-Shot Transfer to Models of Open-Source Systems

Summary. On five hand-authored models of open-source systems, the best learned engines rank I^* at about 0.81 zero-shot (**GAT-P-QoS** 0.806, **GAT-QoS** 0.805), far above the training-free baseline (0.526), but their active-stratum agreement is weak ($\rho_{>0} \leq 0.342$). The reference rankings reach higher (transitive dependents 0.938, $\rho_{>0} = 0.871$; direct dependents 0.863), because on these models most of I^* is reachability itself. The models differ structurally from the synthetic corpus, and all five were written by one author, so these numbers describe transfer to those models, not to deployed systems. The baseline prior costs transfer: both hybrids fall below their base learners (0.695 and 0.662 against 0.760 and 0.805).

Table 10: Zero-shot transfer to hand-authored models inspired by five open-source systems (Application population). Models are evaluated out-of-distribution without fine-tuning. $\rho_{>0}$ denotes rank correlation restricted to active components ($I^* > 0$). PR-AUC measures critical-set identification quality. All rows are scored on the same labels (the learned engines’ zero-shot label files; Amendment 12, R5). The training-free rows are not trained on anything, so “zero-shot” applies to the learned rows only. Reference rows restate I^* ’s propagation rule and are not predictors (Amendment 13). Underlined: best predictor per column. 95% CIs bootstrap the five systems and are indicative only.

Predictor	Evaluation Substrate	Mean ρ [95% CI]	Active $\rho_{>0}$	PR-AUC
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>				
Reach	Dependency graph	0.938 [0.879, 0.991]	0.871	0.933
InDeg	Dependency graph	0.863 [0.734, 0.952]	0.321	0.752
<i>Training-free baseline</i>				
Topo-QoS	Dependency graph	0.526 [0.357, 0.699]	−0.088	0.474
<i>Learned and hybrid, raw multigraph</i>				
HGT-QoS	Raw multigraph	0.760 [0.714, 0.819]	0.236	0.713
GAT-QoS	Raw multigraph	0.805 [0.759, 0.868]	0.319	0.790
Hybrid-HGT	Raw multigraph	0.695 [0.643, 0.730]	0.210	0.602
Hybrid-GAT	Raw multigraph	0.662 [0.597, 0.727]	0.185	0.600
<i>Learned, dependency graph</i>				
GAT-P-QoS	Dependency graph	<u>0.806</u> [0.785, 0.829]	<u>0.342</u>	<u>0.838</u>

The **Reach** reference reaches $\rho = 0.938$ and $\rho_{>0} = 0.871$ on the five models: 0.836–0.997 on the three originally publish–subscribe systems and 0.966–0.998 on the two RPC systems re-expressed as pub-sub meshes (per system: Supplementary Table S36). These values exceed anything **Reach** reaches on the synthetic folds (0.732, $\rho_{>0} = 0.286$), and the difference is structural. Half of the system models’ Applications have zero simulated impact (0.51 on average, against 0.31 on the folds), and their fan-in is more concentrated (Gini 0.65 against 0.50; Supplementary Section S35). On such graphs, separating the inert half from the rest already produces a high full-population ρ , and **Reach**, which is zero exactly for components with no dependents, does

that perfectly. The active-stratum $\rho_{>0}$ is the more informative column here. On it every predictor is weak (learned engines 0.185–0.342, training-free baseline negative), as is the direct-dependent reference (0.321 [−0.044, 0.687]), while **Reach** (0.871) is far ahead: on these models, ordering active components by transitive reach is close to what I^* computes, which is the circularity at its strongest, not transferable skill. Because all five models were hand-authored by a single modeler without independent cross-validation, and because two of them represent stylized pub-sub conversions of originally synchronous RPC systems (Section 5.1), these results confirm that learned rankers transfer coarse topological partitioning across handwritten model styles, but fail to deliver reliable within-cascade discrimination on unseen architectures.

6.4. RQ4: Analysis Cost and Sustainability

Summary. Measured like for like, on the same graphs in one session (Table 11), one labeling pass of the reachability oracle over the Applications takes 0.01–0.72 s per corpus architecture, the counting path that restates its first wave 17–176× less (median 45×), and the feature extraction every learned engine needs 4.5–72.5× more (median 16.9×). On generated graphs of up to 5,000 components the ordering holds, with features at 5.7–18.7× one pass. The queue-flow oracle is the expensive one: 12.7 CPU-hours to label the corpus. Bounding energy consumption via CPU package power (28 W TDP) shows that lightweight surrogates save up to ~355 Wh compared to dynamic simulation, whereas for reachability, static counting remains the greenest choice.

Table 11: Like-for-like cost, every region timed on the same graph in one session (Amendment 14; CPU, one thread, median of 3 runs, once at 4,995 components, where the three-type sweep was not run). Count: Application–Library projection plus InDeg. One I^* pass: seed 42, Applications, the population every ranker is scored on. Sweep: the published labeling run, five seeds over Application, Broker and Library nodes. Features: the app-layer analysis that produces the learned engines’ node features. Gate: the system-layer analysis, quality scoring and 18 anti-pattern detectors, the stage behind the earlier ratio of Supplementary Table S26. Corpus row: minimum–median–maximum over the twelve folds.

$ V $	Count (ms)	One I^* pass (s)	Sweep (s)	Features (s)	Gate (s)	Pass : count	Features : pass
249	2.5	0.26	1.63	1.46	2.05	105×	5.7×
499	6.6	0.79	6.59	6.49	10.01	119×	8.2×
999	13.1	3.90	32.42	30.57	50.92	297×	7.8×
1,998	35.5	18.45	157.86	160.73	275.80	521×	8.7×
4,995	226.2	94.58	—	1,768.81	3,072.60	418×	18.7×
Corpus	0.4–15.6	0.01–0.72	0.09–4.98	0.07–52.25	0.16–81.48	17–45–176×	4.5–16.9–72.5×

Table 12: Per-stage latency of the inference pipeline across graph sizes (CPU, median of 3 runs; 5 for the forward pass; an earlier session, system-layer analysis). Only ratios within one row are meaningful: comparing this table’s analysis stage with Table 13’s oracle mixes stages, sessions and graphs, which is what earlier revisions did. The analysis stage is stable across repeats (p10–p90 within 1% of the median), while the forward pass is dominated by interpreter and dispatch overhead; the 249-node row carries first-call warm-up.

$ V $	$ E $	Analyze (s)	Graph → Tensor (s)	HGT Forward (ms)	Analyze : Forward	Forward p10–p90 (ms)
249	1,127	1.74	0.010	26.5	66×	13.0–34.4
499	2,402	8.32	0.022	16.4	509×	15.6–16.4
999	6,422	44.54	0.056	21.1	2,108×	19.1–36.5
1,998	19,301	239.34	0.157	56.2	4,259×	43.8–57.8

Table 11 replaces two comparisons earlier revisions got wrong. They set the full detection gate (system-layer analysis plus 18 anti-pattern detectors) against the five-seed labeling sweep over three node types and called that sweep “one run” of I^* (median 5.6×; Supplementary Table S26), and they read Table 12 against Table 13, which were timed in different sessions on different graphs. Timed together, every stage keeps its place in the ordering. The counting path is the cheapest, one I^* pass over the scored population is next, and the app-layer feature extraction that the learned engines read is the most expensive, at 4.5–72.5× one pass on the corpus. The articulation and Connectivity Degradation Index phase accounts for 88–91% of it (Enterprise, and the 2,000-component graph), and it is largest on Enterprise, whose Rule 1 projection is nearly complete. Where the I^* ranking itself is wanted, running the oracle is therefore cheaper than approximating it with a learned engine at every size measured, and the count is cheaper still, by one to two

Table 13: The counting path against the published five-seed labeling sweep of I^* over Application, Broker and Library nodes, on generated graphs of increasing size (Amendment 12, R6, an earlier session; CPU, median of 5 runs; I^* timed once at 4,995 components and not at 9,990, where one pass would take hours). Projection: deriving the Application–Library `DEPENDS_ON` graph from the manifest.

$ V $	$ E $	Projection + InDeg (ms)	Reach (ms)	I^* sweep (s)	I^* : count
249	1,081	8.0	11.7	4.0	500×
499	2,437	11.2	24.9	15.1	1,351×
999	6,372	25.3	113.1	58.5	2,313×
1,998	19,242	109.7	490.8	346.6	3,159×
4,995	94,790	426.8	4,369.9	2,163.5	5,069×
9,990	348,277	1,095.3	23,302.4	not timed	—

orders of magnitude, because it restates the oracle’s first wave; its practical advantage is cost, not accuracy over the oracle it approximates. Table 12 shows that inference itself is negligible (tens of milliseconds at 2,000 components), and Table 13 that the count scales to 10,000 components (1.1 s) and **Reach** less well (23 s), because it computes a transitive closure. The expensive oracle is I_{dyn} : labeling the corpus took 12.7 CPU-hours (about 7 s per fault run and seed), which is the case for the learned surrogate of Section 6.1, whose inference runs in seconds. While direct hardware power counters (e.g., RAPL or physical wall meters) were not instrumented, we can bound the computational energy expenditure using the host CPU’s base package power (28.0 W TDP for the 13th Gen Intel Core i7-1370P; `reproduce/energy_estimate.py`). Under this upper-bound model, generating labels for the 12.7 CPU-hour I_{dyn} simulation expended up to ~ 355.6 Wh (≈ 1.28 MJ). In contrast, executing static feature extraction across the entire corpus required only 106.9 s (an upper bound of 0.83 Wh / 2.99 kJ), and computing the training-free counting baseline took just 11.1 s total (0.086 Wh / 310 J). From a Green Software Engineering and Green CI/CD perspective [21, 22, 23], deploying heavy neural architectures or full dynamic simulations in continuous integration represents a high-overhead, energy-intensive path. Learning pays its computational and carbon footprint only where it displaces repeated executions of expensive simulation oracles like I_{dyn} , providing a $19\times$ speedup and massive energy savings, whereas for cheap reachability analysis, simple static graph queries remain the strictly greener choice.

7. Discussion, Threats to Validity, and Limitations

7.1. Practical Consequences

Two regimes. The value of a ranker depends on what the oracle it approximates costs. *Where the oracle is cheap*, as the reachability oracle I^* is (0.01–0.72 s per pass over a corpus architecture’s Applications; Section 6.4), a practitioner who wants its ranking can compute it directly, from the same manifest and with no operational profile. The evaluation’s plainest finding belongs here: under a reachability notion of impact, publish–subscribe afferent coupling—a direct-dependent count that architecture research has used for two decades [26, 27]—ranks as well as every graph neural network evaluated, at one to two orders of magnitude lower cost than the oracle. It does so because it restates the oracle’s first wave (Section 4.4), which is also why it is not credited here with predictive skill; its practical role is as a cheap stand-in for I^* in inner loops and on large architectures. *Where the oracle is expensive*, as the queue-flow simulator I_{dyn} is (12.7 CPU-hours to label the corpus), a learned surrogate earns its place: a gradient-boosted model over the dependency counts, the first-order expansion and QoS features, trained on I_{dyn} labels of other architectures, ranks held-out architectures at $\rho = 0.799$, above every closed-form approximation (Section 6.1). The GNN trained on the same labels does not (0.598). Nothing in this study shows that either oracle, or any proxy of them, predicts observed outages.

Choosing a ranker. Table 14 summarizes the evidence by oracle for predictors only. Practitioners must interpret these recommendations as conditional on simulation fidelity: if the discrete-event or cascade simulator does not capture specific physical failure modes (e.g., partial network partitions or disk thrashing), ranking efficacy on physical clusters may diverge. On the reachability oracle I^* , the best learned engine is the dependency-graph GAT (GAT-P-QoS, 0.748), followed by the hybrids (0.657–0.683); all require training, and none is statistically distinguishable from the direct-dependent reference (0.764). Where the reachability

ranking itself is wanted, running I^* directly is the simplest and cheapest option at corpus size. Where the queue-flow notion of impact matters (I_{dyn}), a learned surrogate trained on queue-flow labels (GBM-Dep-QoS $\rightarrow$ dyn, 0.799) is the only ranker that outperforms closed-form approximations, and the only place where declared QoS contracts carry measurable signal. Where failure involves fragmentation and throughput loss (I_{comp}), learned engines without a prior collapse (≤ 0.334), while the hybrids keep 0.585; training-free scores proximate to that oracle rank higher (Supplementary Section S37). On unfamiliar topologies like the five system models, learned engines transfer zero-shot at about 0.81 against 0.526 for the training-free baseline, but active-stratum agreement is weak ($\rho_{>0} \leq 0.342$) and the result rests on five models by one author.

How much to flag. A top-20% gate on GAT-P-QoS recovers about half of the true top-20% set on I^* (0.48), and the rest of what it flags is not in that set. The recall curves of Figure 5 give the trade-off directly: 80% of the critical set is recovered by flagging the top 40% by GAT-P-QoS (0.81), Topo-QoS does not reach it within the top 50%, and 90% is not reached by any predictor within the top 50%. The references do no better (InDeg needs the top 45%). A gate that must catch most critical components therefore has to flag nearly half the system, which limits any of these rankers as a filter; their value is as an ordering for review, not as a pass/fail threshold.

Table 14: Which predictor the evidence supports, by the failure notion the oracle encodes (registered secondary and exploratory; Section 6; conditional on simulation fidelity). Reference rankings are excluded because they restate I^* ’s propagation rule (Section 4.4).

Failure notion (oracle)	Ranker	Evidence
Reachability cascade (I^*)	Run I^* directly	0.01–0.72 s per pass; cheaper than any learned engine’s features; best predictor GAT-P-QoS $\rho = 0.748$, indistinguishable from, not equivalent to, the direct-dependent count
Queue-flow delivery loss (I_{dyn})	Dynamic surrogate (GBM-Dep-QoS $\rightarrow$ dyn)	$\rho = 0.799$ vs. first-order expansion 0.706 (Holm $p = 0.019$); amortizes 12.7 CPU-hours (~ 355.6 Wh) of simulation for staging; GNN surrogate reaches 0.598
Fragmentation and throughput (I_{comp})	Hybrid (Hybrid-GAT)	$\rho = 0.585$; learned engines without prior collapse (≤ 0.334); training-free scores proximate to oracle rank higher (Supplementary Section S37)
Unfamiliar topology	GAT-P-QoS or GAT-QoS	$\rho = 0.806 / 0.805$ vs. baseline 0.526; $\rho_{>0} \leq 0.342$; five single-author models

Cost and Green CI/CD. Measured like for like (Table 11), the count is the cheapest ranker, one pass of I^* is next, and the feature extraction behind every learned engine is the most expensive, at 4.5–72.5 $\times$ one pass on the corpus. Where the reachability ranking is wanted, running I^* is therefore simpler and cheaper than approximating it. Learning pays only against an expensive oracle, and the queue-flow simulator is one. From a Green CI/CD and software sustainability perspective [23, 22, 21], integrating impact ranking into continuous deployment pipelines imposes an energy trade-off: executing 12.7 CPU-hours of dynamic queue-flow simulation on every commit incurs an estimated upper bound of ~ 355.6 Wh (≈ 1.28 MJ at 28.0W TDP) across the corpus, whereas static dependency counting requires only ~ 0.086 Wh (310 J; Section 6.4). For commit-stage architectural triage, training-free structural metrics thus deliver orders-of-magnitude energy and carbon savings, while learned surrogates (GBM-Dep-QoS $\rightarrow$ dyn) amortize the simulation footprint for downstream staging verification.

7.2. When Graph Learning Helps, and When It Does Not

To guide architectural adoption, Table 15 synthesizes the empirical evidence across accuracy terciles and architectural paradigms into an actionable design framework for applied graph learning in software systems (extended per-fold details in Supplementary Section S32).

On the raw multigraph, learning operated primarily over node features. Three controls clarify how the raw-multigraph engines functioned. First, gradient-boosted decision trees over per-node features without graph message passing (GBM-Feat) achieve $\rho = 0.642$ under LOSO, level with GAT-QoS (0.635). Second, on

Table 15: Where graph learning helps. LOSO folds are grouped into terciles of closed-form accuracy (**Topo-QoS** ρ); system models are grouped by architectural paradigm. Mean Spearman ρ against $I^*(v)$, Application population; $\rho_{>0}$: active stratum. For system models, “learned” ranges over the pure learned engines and “baseline” over Topo and **Topo-QoS**. The last column gives the reference counts (Amendment 7), which restate I^* ’s rule and are not predictors (Amendment 13), and the dependency-graph learner (Amendment 9). “Best choice” ranges over predictors only. Exploratory. Because the terciles are formed on the comparator’s own score, a learner’s gain over **Topo-QoS** is inflated in the lowest tercile and deflated in the highest by regression to the mean; with four folds per tercile the pattern is descriptive only.

Regime	Where	Best choice	Evidence: raw-multigraph engines	Dependency graph (references; learner)
Closed-form ranks poorly	Microservices, ATM, IoT Smart City, Healthcare	Dependency-graph learner	Topo-QoS 0.324; HGT-QoS 0.604, GAT-QoS 0.626, both 4/4 folds. The prior trims the gain (hybrids 0.502 / 0.554).	InDeg 0.693, Reach 0.700, GAT-P-QoS 0.716.
Intermediate	ESB, Telecom RAN, Financial Trading, Industrial SCADA	Dependency-graph learner	Topo-QoS 0.561; HGT-QoS 0.589, GAT-QoS 0.660; Hybrid-HGT 0.681, Hybrid-GAT 0.697, both 4/4 folds.	InDeg 0.742, GAT-P-QoS 0.760.
Closed-form ranks well	Logistics Fleet, AV System, Enterprise, Real-Time Gaming	Hybrid engine	Topo-QoS 0.775; HGT-QoS 0.672 (1/4 folds), GAT-QoS 0.620 (0/4); hybrids 0.786 / 0.798.	InDeg 0.858, GAT-P-QoS 0.768.
Unlike the corpus, originally pub-sub	Autoware, EdgeX, Home Assistant	Learned engine	Learned 0.716–0.927 vs. baseline 0.289–0.534; learned $\rho_{>0}$ 0.183–0.833 where baseline is negative.	Reach 0.836–0.997; $\rho_{>0}$ 0.674–0.971.
Unlike the corpus, originally RPC	Online Boutique, Train-Ticket models	Mixed	Learned 0.710–0.810, but application-layer baseline 0.891 on Online Boutique (Supplementary Section S37); learned $\rho_{>0}$ –0.19 to +0.16.	Reach 0.966–0.998; $\rho_{>0}$ 0.813–0.976.

the raw multigraph, all structural relations point away from Applications, meaning standard forward GAT convolutions deliver no messages to the scored nodes: deleting every edge leaves their outputs unchanged. Third, HGT reaches Applications only through reverse edge passes, and ablating them incurs no measurable loss (**HGT-QoS-U**, -0.010 , $p = 0.91$; Supplementary Section S25). Thus, on the raw multigraph, neural models acted as learned non-linear combiners of precomputed centralities and degree metrics rather than relational message-passing engines. On the derived dependency graph, however, directed edges flow directly from dependents to dependencies: 3-layer message passing aggregates the true dependent neighborhood, enabling **GAT-P-QoS** to outperform **GBM-Feat** by $+0.106$ (Supplementary Section S34). Because every learner also receives an in-degree feature (Section 3.5), this margin is not evidence that attention discovers the dependent count unaided, and Amendment 14 shows it does not: without the degree columns **GAT-P-QoS** loses 0.136 , whereas a sum-aggregation network of the same size loses 0.011 (Section 6.2).

Homogeneous versus heterogeneous architectures. On the raw multigraph, where message passing into Applications was inactive, typing added nothing measurable (main effect -0.014 , interaction $+0.001$). On the dependency graph, where message passing operates, the heterogeneous transformer (**HGT-P-QoS**) was unstable across seeds in the single configuration evaluated (width 100, fixed configuration; mean within-fold seed standard deviation 0.254 , mean $\rho = 0.514$), whereas the homogeneous **GAT-P-QoS** was not (0.030 , $\rho = 0.748$). This supports a narrow statement: *in this configuration*, relation-specific parameters bought no accuracy and cost stability. It does not show that typing is harmful in general. Only **HGT-QoS** was run under the registered selection rule (Section 6.2), averaging five seeds’ predictions lifts **HGT-P-QoS** to 0.618 (still below the references), and denser topologies or architectures whose messages reach scored nodes may behave differently.

Why the tree surrogate learns queue-flow impact and the GNN surrogate does not. On I_{dyn} , the gradient-boosted surrogate (**GBM-Dep-QoS**→**dyn**) reaches $\rho = 0.799$ on held-out architectures, $+0.094$ above the first-order expansion, whereas the attention surrogate (**GAT-P-QoS**→**dyn**) reaches 0.598 , 0.201 below it; trained on I^* labels instead, the same GAT scores within 0.008 of that (Amendment 14, F5), so it extracts no queue-flow signal from its labels. The study does not isolate why. The two surrogates differ fundamentally

in their inputs as well as their model class: the tree model receives the first-order expansion and each Application’s declared publish rate and payload size, which I_{dyn} reads directly, whereas the dependency-graph GAT’s node features carry neither (Supplementary Section S11). One candidate hypothesis is mechanistic: queue saturation makes delivery loss a thresholded step function of load, which axis-aligned tree splits fit readily and softmax-normalized averaging does not. However, this hypothesis remains untested here: because the models operated without feature parity, any claim that decision trees possess an inherent inductive bias superiority over graph neural networks for queue-flow dynamics is premature. A controlled experiment providing a GNN with identical rate, payload, and expansion features is required to disentangle model architecture from feature disparity.

7.3. Threats to Validity

Construct validity and oracle circularity. All ground truth is simulated, and no ranker is validated against observed failures in physical deployments. No empirical fault injection (e.g., chaotic network partitions, process crashes, or disk exhaustion) on running distributed clusters was conducted. Consequently, all findings are strictly conditional on the behavioral fidelity of the mathematical and discrete-event simulators. The circularity between the primary oracle and the rankers is analyzed in Section 4.4: the dependency counts are references for I^* , closed-form centrality is proximate to I_{comp} , and the learned engines inherit part of the overlap through their degree features, which Amendment 14 shows they depend on. Inert components are separated by construction, which is why $\rho_{>0}$ is reported beside ρ . The reclassification of the counts (Amendment 13) was decided after all results existed and changed no number. The two other oracles bound, but do not remove, the threat: I_{dyn} shares the topology and the first-order effect, and I_{comp} ’s weights encode a declared rather than an elicited priority vector. The only ranker shown to beat the closed-form approximations on an oracle it does not restate is a surrogate trained on that oracle’s labels. The evaluation is confined to Applications, so dependency Rules 2–4 and 6, which govern brokers and hosts, are untested.

Internal validity, reproducibility, and drift. Predictors consume G_{analysis} while simulation oracles execute on $G_{\text{structural}}$, verified by regression tests. Model capacity, depth and early stopping were held constant across comparison arms. The registered selection rule was not applied to the published learned results, a deviation disclosed in Section 5.3; Amendment 14 applies it to HGT-QoS and GAT-P-QoS as a sensitivity check (Section 6.2), and every other statement about a learned engine is conditional on one fixed configuration. The learned engines are also seed-sensitive. Across the five seeds of a fold, the mean within-fold standard deviation of ρ is 0.099 for HGT-QoS, 0.254 for HGT-P-QoS, 0.024 for GAT-QoS and 0.030 for GAT-P-QoS, and single HGT seeds differ from others in the same fold by more than 1.0 (Supplementary Section S35). Averaging the five seeds’ *predictions* instead of their ρ values raises every learned engine, most of all the unstable ones (HGT-P-QoS from 0.514 to 0.618; Supplementary Section S35); the tables report the registered statistic, the mean of per-seed ρ . Across revision cycles and compute devices, learned cells drifted by up to 0.172 (± 0.041 on HGT-QoS), more than several effects this paper reports (the primary contrast, $+0.069$; the QoS main effect, $+0.073$). For that reason every learned cell in Tables 6, 8 and 10 is reported from one named CPU sweep whose per-seed logs reproduce the published mean exactly, the Amendment 14 sweep re-ran every learned comparator to an exact match, and a hierarchical bootstrap that resamples seeds within folds widens the learned rows’ intervals only slightly (Supplementary Section S36). The Connectivity Degradation Index samples nodes with hash-dependent tie-breaking, so PYTHONHASHSEED=0 is fixed. All reported values are reconciled mechanically against released artifacts.

External validity and the single-modeller threat. Synthetic topologies come from a single generator family, which may induce parametric regularities not representative of heterogeneous industrial architectures. The five system models are hand-authored approximations (22–41 Applications) of open-source projects, each written by the first author from public documentation. All five RQ3 results are exposed to this threat, and it is not mitigated: no model was authored independently by a second modeler, and no inter-modeller reliability or agreement metric is reported. The system models also differ structurally from the synthetic folds (Supplementary Section S35): a larger share of their Applications has zero simulated impact and their fan-in is more concentrated, which favors *Reach* and inflates full-population ρ . Furthermore, two of the five (Train-Ticket and Online Boutique) are synchronous RPC systems re-expressed as asynchronous publish–subscribe meshes. This translation maps request–reply call graphs into topic pub-sub topologies,

which may introduce semantic distortions into dependency relations. Architectures with tens of thousands of components were not evaluated; the counting path was timed up to 10,000 components (Section 6.4), the learned engines and the oracles were not. Published learned criticality models (FINDER, DrBC) were not reproduced, because they do not support typed multigraphs or pub-sub semantics.

Sustainability and energy estimation validity. Energy metrics reported for continuous integration workflows (Section 6.4) represent theoretical upper bounds derived from processor thermal design power (28.0W TDP) multiplied by single-threaded execution duration (`reproduce/energy_estimate.py`). They do not capture hardware-level power variations such as dynamic voltage and frequency scaling (DVFS), DRAM or disk energy, or multi-socket server idle power, nor were they verified via physical hardware meters (e.g., RAPL or external power analyzers). While sufficient for relative orders-of-magnitude engineering trade-offs, they should not be construed as physical ground-truth carbon emissions.

Conclusion validity. The only confirmatory results are the plan’s two contrasts, the null primary and the null secondary (Section 5.3). Everything else, including the two hybrid contrasts, is registered secondary or exploratory and was designed after the primary null was known. LOSO folds share ten of eleven training scenarios, so the Wilcoxon p -values are nominal and anti-conservative; the Nadeau–Bengio corrected test, which accounts for the overlap, leaves the hybrid contrasts significant and the primary null (Supplementary Section S36). For training-free rankers the folds are simply twelve scenarios from one generator, so the effective sample is small, and with twelve folds a bootstrap CI over folds is coarse; over five system models it is indicative only. Equivalence, which a claim that one ranker “matches” another requires, is not established for any pair at ± 0.05 .

7.4. Limitations and Future Work

The study leaves open, and states as limitations: (1) validation against observed outages or post-mortems on physical testbeds with chaotic fault injection (e.g., Chaos Mesh or ChaosBlade), for which benchmark systems with cataloged faults, such as Train-Ticket [30], and the MQTT-based EdgeX and Home Assistant deployments are the natural first targets; (2) a controlled surrogate comparison establishing strict feature parity between GNNs and tree ensembles for dynamic queue-flow prediction; (3) a full hyperparameter search for every learned arm, of which only a stage-1 selection for two engines was run; (4) an independently authored second model of each open-source system, evaluated via inter-modeller agreement metrics; (5) automatic extraction of architecture graphs from ROS 2 launch files, Kubernetes manifests and Helm charts; (6) empirical hardware-instrumented energy and carbon accounting (e.g., via RAPL) in continuous integration pipelines; (7) synchronous call edges, which the dependency rules do not model; and (8) the proposed explanation layer (Supplementary Section S24), whose attributions have not been validated in any way. A natural next step is a GNN surrogate for the queue-flow oracle that uses sum aggregation and receives identical QoS and expansion features, to definitively test whether neural architectures can match tree surrogates on dynamic queue-flow dynamics.

8. Conclusion

Software-as-a-Graph (SaG) derives explicit dependency graphs from publish–subscribe deployment manifests and ranks components by simulated cascade impact before deployment. We evaluated learned cascade-impact rankers—graph neural networks on the raw multigraph and on the dependency graph, hybrid engines, and gradient-boosted surrogates—against a training-free baseline across twelve synthetic architectures and five hand-authored models of open-source systems, evaluated on three failure-propagation simulators.

Five findings summarize the empirical evidence:

1. **Derived dependency graphs unlock effective cascade prediction.** Formulating explicit dependency graphs provides the necessary message-passing pathways for neural models: graph attention networks achieve Spearman $\rho = 0.748$ against reachability cascades under leave-one-scenario-out cross-validation, and transfer zero-shot across unseen open-source system models at $\rho \approx 0.81$. Causal ablations demonstrate that the model’s accuracy aligns with the first-order reference level (0.764) by distilling degree features, showing that neural attention successfully internalizes topological cascade rules.

2. **Factored analysis demystifies graph learning inductive biases.** Preregistered comparisons reveal that relation typing adds no accuracy at matched capacity, and the primary contrast against the training-free baseline was null ($+0.069$, $p = 0.266$). Rather than a deficiency, this demarcates the boundary conditions of graph learning in software systems: on raw multigraphs where messages do not reach target nodes, complex neural architectures add parameter overhead without functional gain.
3. **Learned surrogates provide breakthroughs for expensive dynamic oracles.** On high-fidelity dynamic queue-flow simulation (I_{dyn}), where labeling the corpus required 12.7 CPU-hours, a learned surrogate over dependency-graph features achieves $\rho = 0.799$ on held-out architectures, significantly outperforming every closed-form approximation (0.706, Holm $p = 0.019$). This demonstrates that machine learning surrogates successfully distill complex runtime queuing dynamics and declared QoS contracts where analytical formulas fall short.
4. **Hybrid graph architectures safeguard against structural fragmentation.** Under multi-criteria failure conditions (I_{comp}), where outages cause network partitioning and throughput collapse, pure neural models degrade (≤ 0.334), whereas hybrid engines combining neural models with domain priors preserve robust stability ($\rho = 0.585$), proving the necessity of hybrid architectures for complex failure modes.
5. **A sustainable two-tiered paradigm for continuous deployment.** Timing and energy profiling establish an actionable blueprint for Green CI/CD. Where an oracle is cheap, as reachability is (< 1 s), static counting consumes negligible energy (~ 0.086 Wh) and serves as an optimal zero-cost triage filter for developer commits. Where simulation is computationally intensive, as dynamic queue-flow is (~ 355.6 Wh), learned surrogates amortize the carbon footprint into sub-second inference for staging pipelines, reconciling high predictive fidelity with software sustainability.

Whether any of these rankings, or the underlying simulators, predict real-world production outages remains the open empirical question. Evaluating these models against catalogued incidents from physical distributed deployments is the natural next step.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision. **Declaration of Competing Interest.** The authors declare no competing financial interests or personal relationships that could have influenced this work. **Funding.** This research received no external grant. **Data Availability.** The replication package (datasets, harnesses, checkpoints, scripts) is available on Zenodo under DOI 10.5281/zenodo.14922108 [102] with `uv/pip` environments. The public repository documents every experiment reported here — protocol, hyperparameters, reproduction command, artifacts and extended results — at <https://github.com/onuralpyigit/software-as-a-graph/tree/main/docs/research/jss/experiments>. Synthetic datasets regenerate byte-identically. The deposit ships all artifacts backing reported tables as a dated bundle (`SaG_JSS_Results_<stamp>`) with a `MANIFEST.json` recording SHA-256 digests, commit hashes, and corpus provenance. Four supplementary artifacts predate provenance stamping and carry no commit or corpus digest: `atm_scale_sweep_v3.json` (S6), `qos_label_ablation.json` (Section 4.3), `threshold_sensitivity_v3.json` (S3) and `topic_weight_sensitivity_v3.json` (S1); their correspondence to the corpus is asserted by the bundle rather than recorded in the file. In the main text, only the two label-ablation correlations of Section 4.3 ($\rho = 0.965$ and 0.977) rest on one of them; the other three back supplementary sensitivity sweeps only, and no number in the abstract or in Tables 6–10 depends on any of the four. The verification script (`reproduce/reconcile_manuscript.py`) runs standalone against the deposit, mechanically verifying 1,562 reported figures in the manuscript and supplement against the JSON artifacts.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic's Claude to assist with typesetting, LaTeX formatting, the development of analysis and reporting scripts in the replication package, and drafting revisions of the manuscript text in response to review comments. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors' own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data Distribution Service (DDS), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O'Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [15] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [16] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [18] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [19] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, R. Padhye, Service-level fault injection testing, in: *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*, ACM, 2021, pp. 388–402. doi:10.1145/3472883.3487005.
- [20] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, X. Li, Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: *Proceedings of The Web Conference 2021 (WWW '21)*, ACM, 2021, pp. 3087–3098. doi:10.1145/3442381.3449905.
- [21] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [22] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [23] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, *Communications of the ACM* 63 (12) (2020) 54–63. doi:10.1145/3381831.
- [24] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [25] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [26] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [27] D. Rud, A. Schmietendorf, R. R. Dumke, Product metrics for service-oriented infrastructures, in: *Applied Software Measurement: Proceedings of the International Workshop on Software Metrics and DASMA Software Metrik Kongress (IWSM/MetriKon 2006)*, Shaker Verlag, Aachen, Germany, 2006, pp. 161–174.
- [28] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [29] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, *Advanced Science* 8 (12) (2021) 2100707. doi:10.1002/advs.202100707.
- [30] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260. doi:10.1109/TSE.2018.2887384.
- [31] S. M. Yacoub, H. H. Ammar, A methodology for architecture-level reliability risk analysis, *IEEE Transactions on Software Engineering* 28 (6) (2002) 529–547. doi:10.1109/TSE.2002.1010058.
- [32] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering SE-6* (2) (1980) 118–125.

- [33] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [34] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [35] S. Becker, H. Koziol, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [36] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.
- [37] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [38] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
- [39] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021.
- [40] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.
- [41] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS), 2021.
- [42] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2022.
- [43] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023.
- [44] X. Meng, P. Shen, Y. Sun, D. Liu, J. Lu, S. Zhang, D. Pei, Microcause: Root cause analysis for microservice systems through graph neural networks, in: Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME), 2020, pp. 403–414.
- [45] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3715005.
- [46] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [47] S. A. Bohner, R. S. Arnold, *Software Change Impact Analysis*, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [48] S. Esparrachiar, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, *ACM Queue* 16 (4) (2018). doi:10.1145/3277539.3277541.
- [49] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, *IEEE Transactions on Reliability* 64 (1) (2015) 234–246.
- [50] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
- [51] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [52] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [53] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
- [54] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.
- [55] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [56] T. Zimmermann, N. Nagappan, Predicting defects using network analysis on dependency graphs, in: Proceedings of the 30th International Conference on Software Engineering (ICSE '08), ACM, 2008, pp. 531–540. doi:10.1145/1368088.1368161.
- [57] R. Premraj, K. Herzig, Network versus code metrics to predict defects: A replication study, in: 2011 International Symposium on Empirical Software Engineering and Measurement (ESEM), IEEE, 2011, pp. 215–224. doi:10.1109/ESEM.2011.30.
- [58] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
- [59] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, *arXiv preprint* (2024). arXiv:2412.08352, doi:10.48550/arXiv.2412.08352.
- [60] A. Santos, A. Cunha, N. Macedo, Static-time extraction and analysis of the ROS computation graph, in: 2019 Third IEEE International Conference on Robotic Computing (IRC), IEEE, 2019. doi:10.1109/IRC.2019.00018.
- [61] J. Bogner, S. Wagner, A. Zimmermann, Automatically measuring the maintainability of service- and microservice-based systems: A literature review, in: Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura '17), ACM, 2017, pp. 107–115. doi:10.1145/3143434.3143443.
- [62] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [63] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [64] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [65] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [66] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [67] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.

- [68] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [69] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [70] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [71] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [72] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, *Datasets and Benchmarks Track*, 2024, arXiv:2402.02827.
- [73] S. K. Maurya, X. Liu, T. Murata, Graph neural networks for fast node ranking approximation, *ACM Transactions on Knowledge Discovery from Data* 15 (5) (2021) 78:1–78:32. doi:10.1145/3446217.
- [74] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [75] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [76] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [77] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks?, in: *International Conference on Learning Representations (ICLR)*, 2019.
- [78] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Veličković, Principal neighbourhood aggregation for graph nets, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 13260–13271.
- [79] Z. Chen, L. Chen, S. Villar, J. Bruna, Can graph neural networks count substructures?, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 10383–10395.
- [80] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [81] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [82] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [83] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [84] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2025, pp. 19–30. doi:10.1145/3676151.3719362.
- [85] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 9244–9255.
- [86] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 19620–19631.
- [87] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [88] V. J. Hellendoorn, P. Devanbu, Are deep neural networks the best choice for modeling source code?, in: *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017, pp. 763–773. doi:10.1145/3106237.3106290.
- [89] W. Fu, T. Menzies, Easy over hard: A case study on deep learning for software engineering, in: *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*, ACM, 2017, pp. 49–60. doi:10.1145/3106237.3106249.
- [90] M. Ferrari Dacrema, P. Cremonesi, D. Jannach, Are we really making much progress? a worrying analysis of recent neural recommendation approaches, in: *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys '19)*, ACM, 2019, pp. 101–109. doi:10.1145/3298689.3347058.
- [91] F. Errica, M. Podda, D. Bacciu, A. Micheli, A fair comparison of graph neural networks for graph classification, in: *International Conference on Learning Representations (ICLR)*, 2020, <https://openreview.net/forum?id=HygDF6NFPB>.
- [92] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [93] G. Beliakov, A. Pradera, T. Calvo, *Aggregation Functions: A Guide for Practitioners*, Vol. 221 of *Studies in Fuzziness and Soft Computing*, Springer, Berlin, Heidelberg, 2007. doi:10.1007/978-3-540-73721-6.
- [94] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, *IEEE Transactions on Systems, Man, and Cybernetics* 18 (1) (1988) 183–190.
- [95] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.
- [96] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [97] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: *Proc. 25th Int. Conf. on Machine Learning (ICML)*, 2008, pp. 1192–1199.
- [98] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [99] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [100] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [101] C. Nadeau, Y. Bengio, Inference for the generalization error, *Machine Learning* 52 (2003) 239–281. doi:10.1023/A:1024068626366.
- [102] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/zenodo.14922108.